

A PROJECT REPORT ON
REAL TIME FRAUD DETECTION SYSTEM FOR ONLINE
FINANCIAL TRANSACTIONS

Submitted in partial fulfilment of the requirements for the award of the degree

Bachelor of Technology
in
COMPUTER SCIENCE & ENGINEERING

Submitted by

C SAILAJA	22G01A0539
A RAJESH	22G01A0510
C RAMA CHARAN	22G01A0537
K LINGAMAIAH	22G05A0570

Under the guidance of

Mrs. M. Malli, M.Tech
Assistant Professor,
Department of CSE



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SRI VENKATESA PERUMAL COLLEGE OF ENGINEERING &
TECHNOLOGY

Accredited by NBA, NAAC, Approved by AICTE AUTONOMOUS),
Permanent Affiliation by JNTUA, Ananthapuramu.

RVS Nagar, Chinnaraja Kuppam, KN Road, Puttur-517583

2025-26





SVPP (AUTONOMOUS)
SRI VENKATESA PERUMAL COLLEGE
OF ENGINEERING & TECHNOLOGY|PUTTUR



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that the project report entitled “**REAL TIME FRAUD DETECTION SYSTEM FOR ONLINE FINANCIAL TRANSACTIONS**” is being submitted by the member of Batch No: **CS22A11**.

C SAILAJA	22G01A0539
A RAJESH	22G01A0510
C RAMA CHARAN	22G01A0537
K LINGAMAIAH	22G05A0570

In partial Fulfillment of the requirements for the degree Bachelor of Technology in Computer Science & Engineering award from Sri Venkatesa Perumal College of Engineering & Technology, Puttur, affiliated to Jawaharlal Nehru Technological University Anantapur, Ananthapuramu. This is the Bonafide work carried out by them under my guidance and supervision during the academic year 2025-2026.

PROJECT GUIDE

Mrs. M Malli M.Tech,
Assistant Professor,
Dept. of CSE.

HEAD OF THE DEPARTMENT

Dr. B. RAMA GANESH, M.Tech, Ph.D., PDF.
Professor & HOD,
Dept. of CSE.

Submitted for the viva-voce examination held on:

Internal Examiner

External Examiner

DECLARATION BY PROJECT GUIDE

I hereby declare that the project report entitled “**REAL TIME FRAUD DETECTION SYSTEM FOR ONLINE FINANCIAL TRANSACTIONS**” is the bonafide work carried out by the members of **Batch No.CS22A11** of Sri Venkatesa Perumal College of Engineering & Technology, Puttur for the award of degree Bachelor of Technology in Computer Science & Engineering during the academic year 2025-2026 is original work and the project has not formed the basis for the award of any degree, diploma, associate fellowship or any other similar title submitted previously.

PROJECT GUIDE

**Mrs. M. Malli, M.Tech.
Assistant Professor,
Dept. of CSE.**

DECLARATION BY PROJECT MEMBERS

We hereby combinedly declare that the project entitled “**REAL TIME FRAUD DETECTION SYSTEM FOR ONLINE FINANCIAL TRANSACTIONS**” submitted by **Batch No. CS22A11** for the award of our degree in B.Tech Computer Science & Engineering in our original work and the project has not formed the basis for the award of any degree, diploma, associate fellowship or any other similar title submitted previously.

C SAILAJA
(22G01A0539)

A RAJESH
(22G01A0510)

C RAMA CHARAN
(22G01A0537)

K LINGAMAIAH
(22G01A0570)

Place:

Date:

ACKNOWLEDGEMENT

The satisfaction and euphoria accompany the successful completion of task and would be incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown all the efforts with success.

We wish to express our deepest sense of gratitude and pay our sincere thanks to our project guide, **Mrs. M. Malli, M.Tech, Assistant Professor**. Department of CSE, who evinced keen interest in our efforts and provided his valuable guidance throughout our project work.

We also express our sincere gratitude to **Dr. B. Rama Ganesh, M.Tech, Ph.D., PDF**. Head of the department of CSE for his great encouragement and valuable support throughout our study.

We owe our gratitude to our principal **Dr. T. Sunil Kumar Reddy, M.Tech., Ph.D.**, for his kind attention and valuable guidance given to us throughout this course.

We sincerely and whole heartedly thank to our beloved **Sri.Ravuri.V.Balaji**, Vice-Chairman for giving art of infrastructure facilities to us throughout our course study and leading to successful completion of our project.

We are very much thankful to our beloved **Dr. R. Venkataswamy**, Chairman of Sri Venkatesa Perumal College of Engineering & Technology, Puttur for his kind attention and valuable guidance to us throughout the course.

We also thankful to all Teaching and Non-Teaching members of CSE Department for helping me to complete this project work by giving valuable suggestions.

We would like to thank the member so four families who assisted in the preparation of this report financially.

The last but not least we express our sincere thanks to all our friends who have supported us in the accomplishment of this project.

Vision and Mission of the Institute

Vision:

To emerge as a center of Excellence for Learning and Research in the domains of Engineering, Technology, Computing and Management

Mission:

Mission	Mission statements
M1	To provide congenial academic ambience with state-of-art resources for learning and research.
M2	Ignite the students to acquire self-reliance in the latest technologies. Unleash
M3	and encourage the innate potential and creativity of students. Include
M4	confidence to face and experience new challenges.
M5	Foster enterprising spirit among students works collaboratively with technical institutes/Universities/Industries of National and International repute.

Vision and Mission of the Department

Vision:

To contribute for the society through excellence in Computer Science and Engineering with a deep passion for wisdom, culture and values.

Mission:

Mission	Mission statements
M1	Provide congenial academic ambience with necessary infrastructure and learning resources.
M2	Inculcate confidence to face and experience new challenges from industry and society.
M3	Ignite the students to acquire self-reliance on the latest technologies. Foster
M4	Enterprising spirit among students.

CO-PO MAPPING

[illegible]

ABSTRACT

The rapid growth of digital wallets and online payment systems has significantly increased the volume of financial transactions conducted over the internet. While these technologies provide speed and convenience, they also introduce security challenges such as unauthorized transactions and financial fraud. Ensuring the safety of online financial transactions has therefore become a critical requirement in modern digital systems.

This project proposes a Real-Time Fraud Detection System based on a rule-based risk analysis approach. The system continuously monitors user transactions by evaluating parameters such as transaction amount, transaction frequency, transaction velocity, and daily transaction limits. Based on these factors, a risk score is calculated for each transaction. Transactions that fall within safe thresholds are processed normally, while transactions that exceed predefined limits or exhibit suspicious patterns are immediately flagged and automatically blocked to prevent potential fraud. The system operates in real time, enabling instant decision-making without manual intervention, and maintains transaction history for monitoring user behavior.

Overall, the proposed system enhances transaction security, reduces financial risk, and improves user trust by preventing fraudulent activities before they occur using efficient and reliable rule-based techniques.

Keywords: Real-Time Fraud Detection, Rule-Based System, Financial Transactions, Risk Scoring, Transaction Monitoring.

LIST OF CONTENTS

CHAPTER 1

INTRODUCTION	01 – 04
1.1 Introduction	01
1.2 Domain Description about the area of project	01
1.3 Problem Definition	02
1.4 Proposed Solution	03
1.5 Objectives	04
1.6 Organization of the Project report	04

CHAPTER 2

LITERATURE SURVEY	05 – 08
2.1 Introduction	05
2.2 Related Work	05
2.3 Overview	08

CHAPTER 3

SYSTEM ANALYSIS	09 – 20
3.1 Introduction	09
3.2 Existing System & Disadvantages	09
3.3 Proposed System & Advantages	11
3.4 Software Requirements	12
3.5 Hardware Requirements	13
3.6 Modules	13
3.6.1 User Registration and Authentication Module	13
3.6.2 Wallet Management Module	14
3.6.3 Transaction Processing Module	15
3.6.4 Real-Time Fraud Detection and Risk Analysis Module	15
3.6.5 Transaction History and Monitoring Module	16
3.7 System Architecture	17

3.8	Feasibility Study	18
3.8.1	Technical Feasibility	18
3.8.2	Economical Feasibility	19
3.8.3	Operational Feasibility	20

CHAPTER 4

SYSTEM DESIGN **21 – 32**

4.1	Introduction	21
4.2	DFD Diagrams	21
4.3	UML Diagrams	23
4.3.1	Use Case Diagram	23
4.3.2	Class Diagram	24
4.3.3	Sequence Diagram	27
4.3.4	Activity Diagram	29
4.3.5	ER Diagram	30

CHAPTER 5

SYSTEM CODING & TESTING **33 – 51**

5.1	Introduction	33
5.2	Implementation	33
5.3	About Java	34
5.3.1	History of Java	35
5.3.2	Advantages of Java	35
5.3.3	JDBC	36
5.3.4	JSP	36
5.3.5	Servlets	36
5.4	Sample Code	37
5.5	Software Testing Strategies	46
5.6	Software Testing Techniques	47
5.6.1	Unit Testing	47

5.6.2 Integration Testing	48
5.6.3 Validation Testing	49
5.6.4 System Testing	50
Results	52 – 56
Introduction	52
Output Screens	53
CHAPTER 6	
CONCLUSION & FUTURE ENHANCEMENTS	57 – 60
6.1 Conclusion	57
6.2 Future Enhancements	59
REFERENCES	61
PO MAPPING	62

LIST OF FIGURES

S.No.	Figure No.	Diagram	Page No.
1	3.1	System Architecture	17
2	4.1	DFD Diagram	22
3	4.2	Use case Diagram for Overall Project	24
4	4.3	Class Diagram for Overall Project	25
5	4.4	Sequence Diagram for Overall Project	28
6	4.5	Activity Diagram for Client	29
7	4.6	ER Diagram	31

LIST OF OUTPUT SCREENS

S.No.	Figure No.	Fig Name	Page No.
1	Screen 1	Registration Page	53
2	Screen 2	Login Page	53
3	Screen 3	Home Page	54
4	Screen 4	Sending Money	54
5	Screen 5	Payment Conformation	55
6	Screen 6	Transaction History	55
7	Screen 7	Money Wallet	56
8	Screen 8	Payment Blocked	56

LIST OF TABLES

S.NO.	TABLE NO.	TABLE NAME	PAGE NO.
1	5.1	Test Case Result	51
2	1	PO Mapping	62

LIST OF ABBREVIATIONS

UML	Unified Modeling Language
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
JS	Java Script
XML	Extensible Markup Language
SQL	Structured Query Language
RDBMS	Relational Database Management System
JDBC	Java Database Connectivity
JSP	Java Server Pages
JEE	Java Enterprise Edition

CHAPTER – 1

INTRODUCTION

1. INTRODUCTION

1.1. Introduction

In recent years, digital payment systems and online financial platforms have become an essential part of everyday life. With the increasing adoption of digital wallets, UPI services, and online banking, the number of financial transactions conducted over the internet has grown significantly. While these technologies offer convenience, speed, and accessibility, they also introduce serious security challenges, particularly the risk of fraudulent transactions and financial cybercrime. Traditional fraud detection systems used in banking and financial applications mainly rely on static rules and manual verification processes. These approaches are often slow, inefficient, and unable to adapt to evolving fraud techniques. As a result, many fraudulent transactions are detected only after financial loss has already occurred, which negatively impacts both users and financial institutions.

To overcome these limitations, this project proposes the development of a Real-Time Fraud Detection System for Online Financial Transactions using Java-based web technologies including JSP, Servlets, and MySQL for developing a complete transaction processing and fraud detection system. The system is designed to monitor and analyze financial transactions instantly using a rule-based approach. Each transaction is evaluated based on parameters such as transaction amount, transaction frequency, transaction velocity, and daily spending limits. Based on predefined rules, a risk score is calculated to determine whether a transaction is safe or potentially fraudulent. The system automatically approves safe transactions and blocks high-risk transactions in real time, ensuring secure and efficient online financial operations.

Overall, this project aims to provide a robust, scalable, and rule-based secure solution that strengthens the security of online financial systems, increases user trust, and supports the growing demand for safe digital transactions.

1.2. Domain Description

The domain of this project lies in the Financial Technology (FinTech) and Digital Payment Systems, specifically focusing on secure online transactions and fraud prevention. This domain involves managing digital wallet operations,

processing financial transactions, monitoring user activity, and ensuring the security of online payments. It plays a crucial role in modern financial systems where users rely heavily on internet-based transactions for daily activities.

In the FinTech domain, ensuring transaction security is essential to prevent financial loss and maintain user trust. Financial systems must handle large volumes of transaction data and monitor user activity in real time to detect suspicious behaviour. With the increasing use of digital wallets, UPI, and online banking, there is a strong need for efficient rule-based fraud detection mechanisms that can quickly identify abnormal transactions based on predefined conditions such as transaction limits and frequency.

The proposed system falls under the category of secure digital payment and fraud detection systems, which aim to enhance the safety and reliability of online financial transactions. This system contributes to improving transaction transparency, reducing fraud risks, and strengthening trust in digital payment platforms.

1.3. Problem Definition

Digital payment systems and online financial platforms process a large number of transactions every day. While these systems focus on providing fast and convenient services, they often lack efficient mechanisms to detect fraudulent activities in real time. Traditional fraud detection methods mainly rely on static rules and manual verification, which are slow and unable to identify advanced or evolving fraud patterns.

Users frequently become victims of unauthorized transactions, high-value frauds, and suspicious payment activities due to delayed detection. Existing systems are not capable of analysing transaction behaviour such as frequency, velocity, and spending patterns effectively. As a result, fraudulent activities are often identified only after financial loss has occurred, leading to reduced user trust and increased risk for financial institutions.

Therefore, there is a need for a robust and intelligent system that can monitor transactions in real time, analyse user behaviour, and detect suspicious activities instantly. The system should be capable of preventing fraud before it occurs by automatically blocking or flagging high-risk transactions, ensuring secure and reliable online financial operations

1.4. Proposed Solution

To overcome the limitations of existing systems, the proposed solution is the development of a Real-Time Fraud Detection System for Online Financial Transactions using a Java-based web technologies including JSP, Servlets, and MySQL for developing a complete transaction processing and fraud detection system. This system provides a secure and intelligent platform that integrates digital wallet functionality with real-time fraud detection mechanisms.

The system allows users to:

- Register and login securely
- Add money to their digital wallet
- Perform online transactions such as sending money
- View transaction history and wallet balance

The core feature of the system is the Fraud Detection Engine, which analyses each transaction in real time using parameters such as:

- Transaction amount.
- Transaction frequency
- Transaction velocity
- Daily transaction limits

Based on these parameters, the system calculates a risk score for every transaction. Depending on the risk level, the system automatically decides whether to:

- Approve the transaction
- Flag it for further verification Block the transaction immediately

The system is implemented as a dynamic web application using modern technologies such as:

- JSP (Java Server Pages) – for interactive UI
- Servlets – for backend logic and request handling
- JDBC – for database connectivity
- MySQL/Oracle – for storing user and transaction data

- HTML, CSS, JavaScript – for responsive front-end

The system also ensures security through authentication, authorization, and secure data handling mechanisms. Overall, this solution improves transaction safety, reduces fraud risk, and enhances user trust in digital payment systems.

1.5 Objectives

- The main objectives of the proposed system are:
- To develop a secure and user-friendly digital wallet system
- To monitor financial transactions in real time
- To detect fraudulent activities based on transaction behavior
- To reduce financial losses caused by unauthorized transactions
- To automate fraud detection without manual intervention
- To improve transaction security and reliability
- To analyse transaction patterns such as amount, frequency, and velocity
- To enhance user trust in online financial systems

1.6 Organization of the Project

The project report is organized into several chapters to provide a clear understanding of the system:

- **Chapter 1:** Introduction – Overview of the project, domain description, problem definition, proposed solution, and objectives.
- **Chapter 2:** Literature Survey – Discussion of existing systems, research papers, and technologies related to fraud detection.
- **Chapter 3:** System Analysis and Design – Includes system requirements, UML diagrams (use case, class, sequence, activity), and system architecture.
- **Chapter 4:** Implementation – Describes the technologies used and the step-by-step development of the system.
- **Chapter 5:** Testing and Results – Explains testing methods, system performance, and output results.
- **Chapter 6:** Conclusion and Future Enhancement

CHAPTER –2

LITERATURE SURVEY

2. LITERATURE SURVEY

2.1 Introduction

A literature survey is a systematic analysis of existing research works and methodologies related to a specific domain. It helps in understanding previously developed systems, identifying their strengths and limitations, and discovering research gaps. This process provides a strong foundation for designing more efficient and improved solutions.

In the domain of digital payment systems and financial security, literature surveys mainly focus on fraud detection techniques, transaction monitoring systems, and risk analysis models. Traditional fraud detection approaches rely on rule-based systems and manual verification, which are often inefficient and unable to adapt to evolving fraud patterns. With the rapid increase in online transactions, there is a growing need for intelligent systems that can detect fraud in real time.

Recent advancements in machine learning, data analysis, and web technologies have enabled the development of more advanced fraud detection systems. Techniques such as behaviour analysis, anomaly detection, and risk-based scoring are widely used to identify suspicious transactions. This section reviews important research works related to fraud detection in financial systems, highlighting their contributions, methodologies, and limitations, which form the basis for the proposed system.

2.2 Related Work

[1] Implementing a Java Microservice for Credit Fraud Detection Using Machine Learning (2024)

Teodoras et al. proposed a fraud detection system using a Random Forest machine learning algorithm integrated into a Java-based microservice architecture. The system analyses credit card transactions in real time and classifies them as legitimate or fraudulent.

- Key contributions include:
- Use of Random Forest for high accuracy in fraud detection
- Handling imbalanced datasets using preprocessing techniques like SMOTE

- Integration of the model into a scalable Java microservice using REST APIs
- Real-time transaction analysis and classification

The results show high accuracy, precision, and recall in detecting fraudulent transactions. However, the system mainly focuses on machine learning and microservices architecture and does not include a complete digital wallet system with user-friendly interfaces, which is addressed in our proposed system.

[2] Credit Card Fraud Detection Using Machine Learning Techniques (Various Studies)

Several studies have explored the use of machine learning algorithms such as Logistic Regression, Decision Trees, Random Forest, and Gradient Boosting for fraud detection. These models analyse transaction features like amount, time, and location to identify unusual patterns.

Key findings include:

- Random Forest provides better performance compared to other algorithms
- Data preprocessing and feature selection improve model accuracy
- Machine learning models can detect complex fraud patterns effectively

According to the *performance table on page 3* of the research paper, Random Forest achieved the highest accuracy (~95%+) compared to other models.

However, most of these studies focus only on model training and evaluation, without integrating the system into a real-time web-based application.

[3] Real-Time Fraud Detection in Financial Systems (E-Commerce & Banking Studies)

Recent research highlights the importance of real-time fraud detection systems due to the rapid growth of online transactions and increasing cyber threats. These systems analyse large volumes of transaction data instantly to detect suspicious activities.

Key contributions include:

- Use of real-time data processing for immediate fraud detection
- Reduction of financial loss through early identification of fraud
- Improved security and reliability in digital payment systems

As discussed in the research paper (*page 2*), fraud in e-commerce is rapidly increasing, creating a strong need for advanced detection systems.

However, many existing systems lack proper integration with user applications and do not provide automated decision-making (approve/block) at the application level.

[4] Machine Learning-Based Fraud Detection with Data Preprocessing Techniques

Some research focuses on improving fraud detection accuracy by applying data preprocessing techniques such as removing irrelevant features, handling missing values, and balancing datasets. These techniques help improve model performance and reduce false positives.

Key contributions include:

- Feature selection to remove unnecessary data (e.g., IDs, names)
- Data cleaning and normalization for better model training
- Improved prediction accuracy and efficiency

The study shows that preprocessing plays a crucial role in enhancing fraud detection models. However, it does not focus on real-time system implementation or integration with a full-stack application.

[5] Microservice-Based Financial Fraud Detection Systems

Modern systems use microservice architecture to build scalable and flexible fraud detection solutions. These systems allow easy integration with existing financial platforms and support real-time processing.

Key contributions include:

- Scalable architecture using REST APIs
- Easy integration with financial services
- Real-time fraud detection capabilities

As explained in the research paper (*page 4*), microservices can process transactions via APIs and return fraud classification instantly.

However, these systems often focus only on backend services and lack a complete user interface and digital wallet functionality.

2.3 Overview

From the above literature, it is evident that significant advancements have been made in fraud detection systems using machine learning algorithms, data analysis techniques, and real-time processing models. Various studies have focused on improving fraud detection accuracy, reducing false positives, and handling large volumes of transaction data efficiently.

Key Observations from Literature:

- Machine learning models improve fraud detection accuracy by identifying complex transaction patterns
- Data preprocessing and feature selection enhance model performance
- Random Forest and similar algorithms provide better results for fraud classification
- Real-time analysis is essential for preventing financial loss before it occurs

Identified Research Gaps:

- Lack of complete integration between fraud detection and digital wallet systems
- Absence of user-friendly full-stack web applications
- Limited focus on real-time automated decision-making (approve/block transactions)
- Many models are complex and not easily deployable in real-world systems

Although many existing systems use machine learning techniques for fraud detection, this project focuses on a rule-based approach due to its simplicity, fast execution, and ease of implementation. The proposed system emphasizes real-time decision-making using predefined rules rather than complex predictive models, making it more practical for small-scale and academic implementations.

CHAPTER – 3

SYSTEM ANALYSIS

3. SYSTEM ANALYSIS

3.1 Introduction

System Analysis is a crucial phase in the Software Development Life Cycle (SDLC) that involves studying the existing system, identifying its limitations, and defining the requirements for a more efficient and secure system. It focuses on understanding user needs, system functionalities, data flow, and performance requirements to ensure the development of a reliable and scalable solution.

In the domain of digital payment systems, system analysis plays an important role due to the increasing complexity of handling financial transactions and ensuring their security. Traditional financial systems often rely on static rule-based mechanisms and manual verification, which are inefficient in detecting modern fraud patterns. These systems lack real-time monitoring, adaptability, and proper integration of fraud detection mechanisms, leading to increased financial risks and reduced user trust.

The proposed Real-Time Fraud Detection System for Online Financial Transactions is designed based on a detailed analysis of these challenges. The system aims to monitor transactions in real time, analyse user behaviour, and detect suspicious activities instantly using risk-based evaluation. It integrates fraud detection with digital wallet functionality, providing features such as secure authentication, transaction processing, and automated decision-making (approve, flag, or block transactions). By utilizing Java Full Stack technologies and a structured approach, the system ensures improved security, efficiency, and reliability in online financial operations.

3.2 Existing System & Disadvantages

Existing System

In current digital payment systems and online financial platforms, transactions are primarily processed with a focus on speed and convenience rather than security. Most existing systems use basic rule-based fraud detection mechanisms, such as predefined limits or simple conditions (e.g., transaction amount exceeding a threshold). These systems are often static and do not adapt to changing fraud patterns.

In many cases, fraud detection is performed after the transaction is completed, through manual review or delayed analysis. This reactive approach results in fraud being detected only after financial loss has already occurred. Additionally, many systems do not analyse user behaviour patterns such as transaction frequency, velocity, or spending habits in real time.

Some advanced fraud detection systems exist, but they are often limited to backend processing and are not integrated with user-facing applications like digital wallets. This lack of integration leads to inefficiencies and reduces the overall effectiveness of fraud prevention mechanisms.

Disadvantages of Existing System

- **No Real-Time Detection:**
Fraud is often detected after the transaction is completed, leading to financial loss.
- **Static Rule-Based Systems:**
Fixed rules cannot adapt to new and evolving fraud techniques.
- **High False Positives:**
Legitimate transactions may be incorrectly flagged as fraud.
- **Manual Verification Dependency:**
Requires human intervention, making the process slow and inefficient.
- **Lack of Behavioural Analysis:**
Does not analyse transaction patterns like frequency, velocity, and user habits.
- **Delayed Response:**
Slow detection increases risk and reduces system reliability.
- **Lack of Integration:**
Fraud detection is not fully integrated with digital wallet systems.
- **Poor User Experience:**
Users face inconvenience due to unnecessary transaction delays or failures.
- **Limited Scalability:**
Traditional systems struggle to handle large volumes of transactions efficiently.

3.3 Proposed System & Advantages

Proposed System

The proposed system is a Real-Time Fraud Detection System for Online Financial Transactions, designed as a dynamic web-based application using Java-based web technologies including JSP, Servlets, and MySQL for developing a complete transaction processing and fraud detection system. The system integrates digital wallet functionality with an intelligent fraud detection engine to provide a secure and efficient financial transaction platform.

Users can register, log in securely, add money to their wallet, and perform transactions such as sending money. The system continuously monitors and analyzes each transaction in real time using parameters such as transaction amount, frequency, velocity, and daily transaction limits. Based on these factors, a risk score is calculated to determine whether the transaction is safe or suspicious.

The system includes modules such as user authentication, wallet management, transaction processing, fraud detection engine, and transaction history monitoring. It ensures seamless communication between system components and provides automated decision-making by classifying transactions as approved, flagged, or blocked instantly.

Advantages of Proposed System

- **Real-Time Fraud Detection:**

Detects suspicious transactions instantly before financial loss occurs.

- **Risk-Based Analysis:**

Evaluates transactions using multiple behavioural parameters.

- **Automated Decision-Making:**

Transactions are automatically approved, flagged, or blocked.

- **Improved Security:**

Prevents unauthorized transactions and protects user accounts.

- **Reduced Financial Loss:**

Early detection minimizes fraud-related risks.

- **User-Friendly Interface:**

Simple and interactive UI improves user experience.

- **Integrated Digital Wallet:**

Combines transaction processing and fraud detection in one platform.

- **Efficient Data Management:**

Stores transaction history securely for monitoring and analysis.

- **Scalability:**

Handles a large number of transactions efficiently.

- **Automation of Processes:**

Reduces manual verification and improves system efficiency.

- **Enhanced User Trust:**

Provides a safe and reliable digital payment environment.

3.4 Software Requirements

Software requirements define the technologies and tools required to develop and run the system efficiently.

Development Technologies

- **Programming Language – Java:**

Used for implementing core backend logic and fraud detection processing.

- **Front-End Technologies – HTML, CSS, JavaScript:**

Used to design responsive and interactive user interfaces.

- **JSP (Java Server Pages):**

Used for dynamic content generation and UI integration.

- **Servlets:**

Handle request processing, session management, and business logic.

- **JDBC (Java Database Connectivity):**

Used for connecting and interacting with the database.

- **Database – MySQL / Oracle:**
Stores user details, wallet data, and transaction history.
- **Fraud Detection Logic:**
Rule-based risk scoring mechanism using predefined thresholds
- **Web Server – Apache Tomcat:**
Used to deploy and run the web application.

Software Environment

- Operating System: Windows / Linux / macOS
- IDE: Eclipse / Eclipse Enterprise Edition / IntelliJ / Spring Boot (optional)
- Browser: Google Chrome / Mozilla Firefox
- Version Control (Optional): Git

3.5 Hardware Requirements

Hardware requirements define the physical components required to develop and run the system.

Minimum Requirements

- Processor: Intel Core i3 or equivalent
- RAM: 4 GB
- Storage: 500 GB HDD
- System Type: 64-bit Operating System

3.6 Modules

The system is divided into multiple modules, each responsible for a specific functionality. The modular design improves maintainability, scalability, and clarity of the system.

3.6.1 User Registration and Authentication Module

The User Registration and Authentication Module is a fundamental component of the system that ensures secure access to the application. It allows users

to create an account, log in using valid credentials, and access the system functionalities safely.

During the registration process, users are required to provide necessary details such as name, email, mobile number, and password. The system validates the input data and stores it securely in the database. Proper validation techniques are applied to ensure data accuracy and prevent invalid or duplicate entries.

The authentication process enables registered users to log in using their credentials. The system verifies the entered username and password with the stored data in the database. If the credentials are valid, access is granted; otherwise, the system displays an error message and restricts access. Session management techniques are used to maintain user login status securely.

To enhance security, the module can include features such as password encryption, input validation, and protection against unauthorized access. This ensures that only authenticated users can perform financial transactions and access sensitive information.

Overall, this module provides a secure and reliable mechanism for user access control, forming the foundation for all other system functionalities

3.6.2 Wallet Management Module

The Wallet Management Module is responsible for managing all activities related to the user's digital wallet. It provides functionality for users to add money, view their wallet balance, and maintain control over their funds in a secure manner. This module acts as the financial backbone of the system, enabling smooth and reliable transactions.

Users can add money to their wallet through predefined options, and the system updates the wallet balance instantly after each operation. It ensures that all wallet operations are performed securely and accurately. The module also prevents users from performing transactions if sufficient balance is not available, thereby maintaining financial integrity.

Additionally, the system keeps track of all wallet-related activities, such as deposits and deductions. Proper validation and security mechanisms are implemented

to protect sensitive financial data. Overall, this module ensures efficient fund management, enhances user experience, and supports seamless digital transactions.

3.6.3 Transaction Processing Module

The Transaction Processing Module is responsible for handling all types of financial transactions within the system. It manages operations such as sending money from one user to another, validating transaction details, and updating account balances accordingly.

When a user initiates a transaction, the system first verifies critical details such as available wallet balance, recipient information, and transaction limits. This ensures that only valid transactions are processed. After validation, the transaction is forwarded to the fraud detection module for further analysis.

Once the fraud detection process is completed, the transaction is either approved or rejected. If approved, the system updates both sender and receiver balances in real time. If rejected, the user is notified accordingly. This module ensures accuracy, consistency, and secure handling of financial transactions, making it a key component of the system.

3.6.4 Real-Time Fraud Detection and Risk Analysis Module

The Real-Time Fraud Detection and Risk Analysis Module is the core component of the system. It evaluates each transaction in real time using predefined rules based on parameters such as transaction amount, transaction frequency, transaction velocity, and daily transaction limits. A risk score is calculated for every transaction, and based on this score, the system decides whether to approve or block the transaction. This rule-based approach ensures fast execution, easy implementation, and reliable fraud detection without the need for complex algorithms.

The module analyses transaction attributes such as transaction amount, frequency of transactions, transaction velocity (number of transactions within a short time), and daily transaction limits. Based on these factors, the system calculates a risk score for each transaction.

If the calculated risk score is within the safe range, the transaction is allowed to proceed. However, if the risk score exceeds a predefined threshold, the system

either flags the transaction for review or blocks it immediately. This real-time decision-making capability helps in preventing fraud before it occurs.

This module significantly enhances the security of the system by reducing financial risks, minimizing false transactions, and ensuring safe digital payments. It also improves user trust by providing a reliable and intelligent fraud detection mechanism

3.6.5 Transaction History and Monitoring Module

The Transaction History and Monitoring Module is responsible for maintaining a detailed record of all transactions performed within the system. It stores important information such as transaction ID, date and time, amount, sender and receiver details, and transaction status.

Users can access their transaction history at any time, which improves transparency and allows them to track their financial activities easily. This feature helps users identify any unusual or unauthorized transactions quickly.

In addition, this module supports monitoring and analysis of transaction data. Suspicious transactions can be reviewed for further investigation, and historical data can be used to improve fraud detection accuracy. It also plays a vital role in auditing and reporting purposes.

Overall, this module enhances system transparency, supports decision-making, and strengthens the overall security framework by providing valuable transaction insights.

3.7 System Architecture

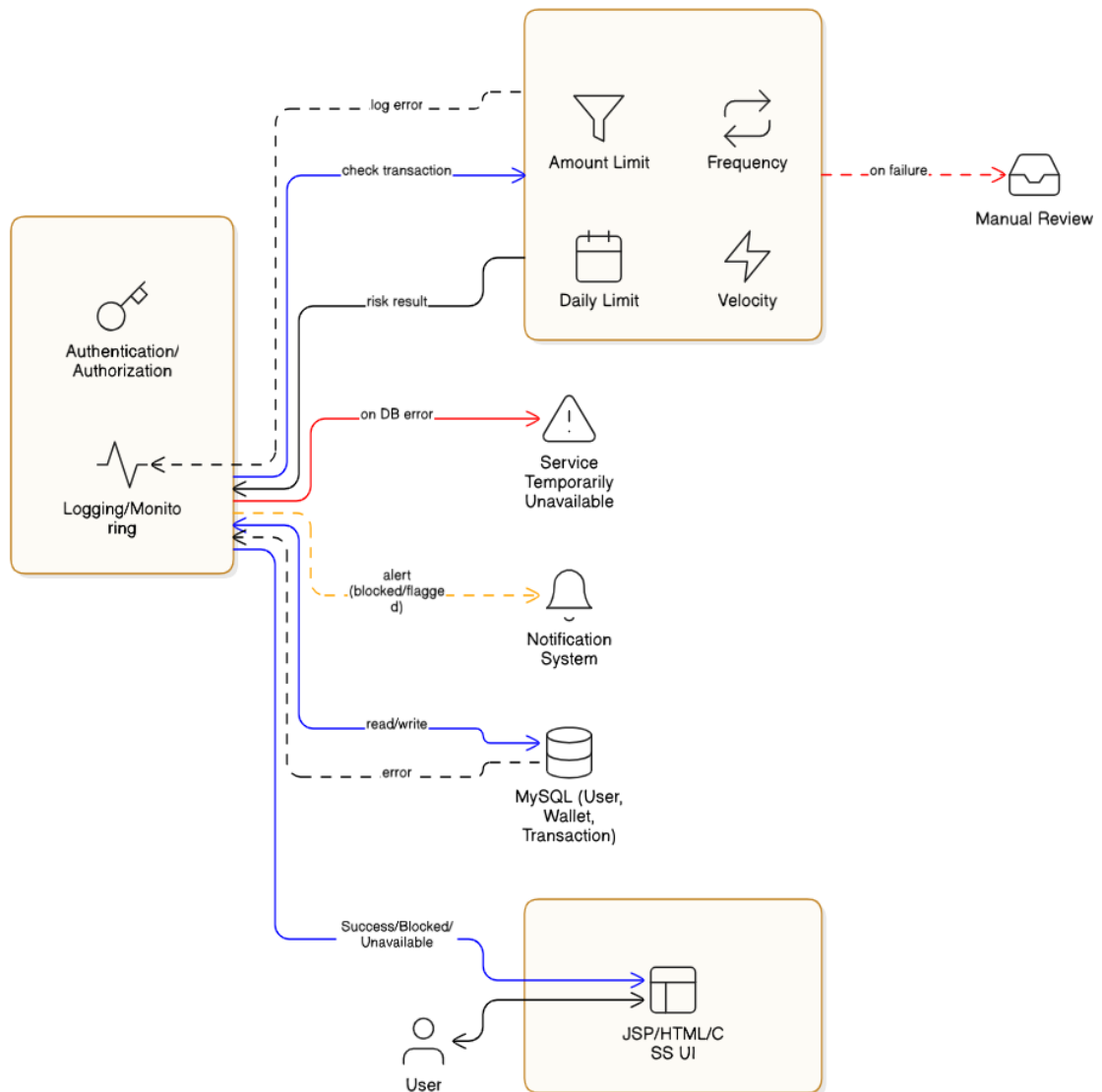


Fig.3.1: System Architecture

3.8 Feasibility Study

The feasibility study evaluates whether the proposed system is practical, efficient, and beneficial for real-world implementation. It analyses different aspects such as technical requirements, cost effectiveness, and operational usability to ensure that the system can be successfully developed and deployed.

The proposed Real-Time Fraud Detection System for Online Financial Transactions has been analysed under three major feasibility categories: Technical Feasibility, Economical Feasibility, and Operational Feasibility.

3.8.1 Technical Feasibility

Technical feasibility assesses whether the required technologies, tools, and resources are available to develop and implement the system effectively.

Key Points:

- The system is developed using Java Full Stack technologies (JSP, Servlets, JDBC), which are widely used, stable, and reliable
- Required tools such as:
 - o Apache Tomcat (Web Server)
 - o MySQL / Oracle Database
 - o IDEs (Eclipse / IntelliJ)
- are easily available and well-supported
- The fraud detection mechanism (rule-based or optional ML-based) can be implemented efficiently using existing programming techniques
- The system can handle real-time transaction processing using standard web technologies
- The application can run on standard hardware configurations without requiring high-end infrastructure
- Developers have sufficient knowledge, resources, and documentation to build and deploy the system

Conclusion (Technical Feasibility):

The proposed system is technically feasible as it uses well-established technologies, requires minimal infrastructure, and can be implemented efficiently with available resources.

3.8.2 Economical Feasibility

Economic feasibility evaluates the cost-effectiveness of developing and maintaining the system.

Key Points:

- The system uses open-source technologies, such as:
 - o Java
 - o MySQL
 - o Apache Tomcat
- No requirement for expensive licenses or proprietary software
- Development cost is low as the system can be built using existing tools and resources
- Reduces operational costs by:
 - o Minimizing manual verification processes
 - o Reducing fraud-related financial losses
 - o Automating transaction monitoring and decision-making
- Early fraud detection helps prevent financial loss, saving money for both users and financial institutions

Conclusion (Economical Feasibility):

The system is economically feasible as it minimizes development and operational costs while significantly reducing financial risks and improving system efficiency.

3.8.3 Operational Feasibility

Operational feasibility evaluates how effectively the system will function in a real-world environment and how easily users can adopt it.

Key Points:

- The system provides a user-friendly interface for performing transactions and monitoring activities
- Minimal training is required for users due to its simple and intuitive design
- Supports both technical and non-technical users
- Automates fraud detection and transaction processing, reducing manual effort
- Provides real-time monitoring and instant decision-making (approve, flag, block)
- Enhances user trust by ensuring secure and reliable transactions
- Improves overall workflow efficiency in digital payment systems
- Can be easily integrated into existing financial platforms and services

Conclusion (Operational Feasibility):

The proposed system is operationally feasible as it is easy to use, improves efficiency, reduces manual work, and provides a secure environment for online financial transactions.

CHAPTER - 4

SYSTEM DESIGN

4. SYSTEM DESIGN

4.1 Introduction

System Design is an important phase in the Software Development Life Cycle (SDLC) that focuses on defining the overall architecture, components, modules, interfaces, and data flow of the system. It transforms the requirements gathered during system analysis into a structured design that can be implemented efficiently.

In this project, the Real-Time Fraud Detection System for Online Financial Transactions is designed using modelling techniques such as Data Flow Diagrams (DFD) and UML diagrams. These diagrams help in understanding how data flows within the system, how different components interact, and how transactions are processed securely.

The design ensures modularity, scalability, and efficient communication between the user interface, backend processing modules, fraud detection engine, and database systems. It also ensures that each transaction is analysed in real time before being processed, thereby improving system security and reliability.

4.2 DFD Diagram

The Data Flow Diagram (DFD) represents how data moves through the system and how it is processed, stored, and utilized by different components.

In the provided DFD, the system consists of key processes such as Transaction Request, User Verification, Risk Analysis, and Payment Processing.

DFD Explanation

- The User acts as the external entity who interacts with the system by providing login credentials and payment details.
- The process begins with a Transaction Request, where the user initiates a payment by entering details such as amount and user ID.
- The system verifies the user by sending the request to the User Database, ensuring that the user is authenticated before proceeding further.
- After verification, the transaction details are forwarded to the Risk Analysis Module, which evaluates the transaction using parameters like amount, frequency, and behaviour patterns.

- The Risk Analysis Module generates a result (Approve or Block) based on the calculated risk score.
- This result is then sent to the Payment Processing Module, which decides whether to continue or stop the transaction.
- If the transaction is approved:
 - The system updates the user's balance in the Wallet Database
 - The transaction details are stored in the Transaction Database
- If the transaction is blocked:
 - The payment is stopped immediately
 - The user is notified about the failed transaction
- Finally, the system sends the Payment Status (Success / Blocked) back to the user.

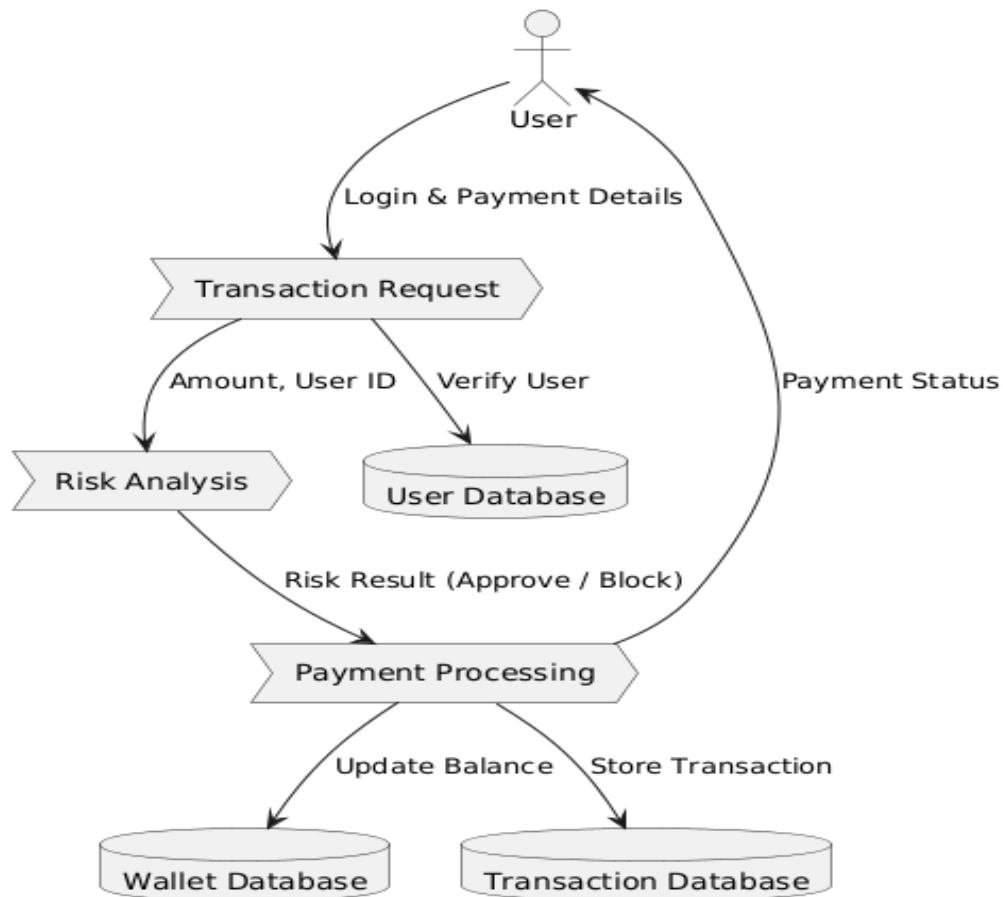


Fig.4.1: DFD (Data Flow Diagram) Diagram

Overall, the DFD clearly illustrates how data flows between users, system processes, and databases, ensuring efficient data management and communication.

4.3 UML Diagrams

Unified Modelling Language (UML) diagrams are used to visually represent the structure and behaviour of the system. These diagrams help in understanding how different components interact and how the system functions internally. The following UML diagrams are used in this project:

4.3.1 Use Case Diagram

The Use Case Diagram represents the interaction between the user (actor) and the system functionalities.

In this system, the primary actor is the User, who interacts with the Real-Time Fraud Detection Wallet System.

The user can perform the following actions:

- Register a new account
- Log in securely
- Add money to the wallet
- View wallet balance
- Send money to another user
- View transaction history

When the user initiates a Send Money operation, the system performs a Fraud Check before processing the transaction. Based on the fraud detection result:

- If the transaction is safe → Approve Transaction
- If the transaction is suspicious → Block Transaction

This diagram clearly shows that fraud detection is an integral part of transaction processing. It defines the system functionalities from the user's perspective and highlights how security is enforced during financial operations.

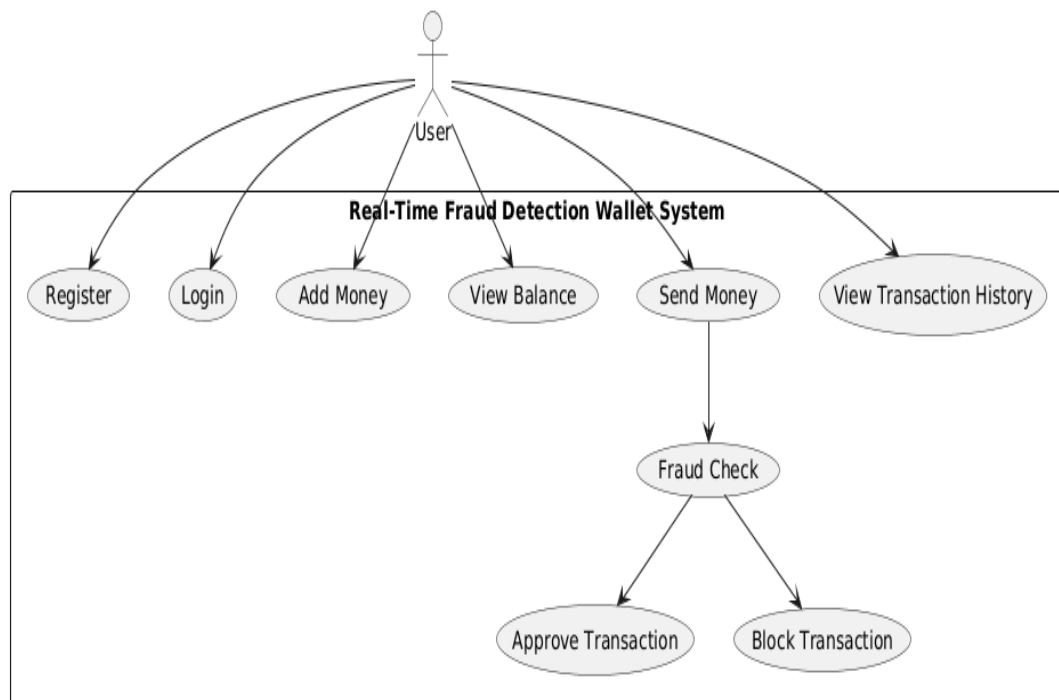


Fig.4.2: Use Case Diagram

4.3.2 Class Diagram

The Class Diagram represents the static structure of the system, including classes, attributes, methods, and relationships.

The main classes in the system are:

User Class

Attributes:

- userId: int
- username: String
- email: String
- mobile: String
- password: String

This class stores user-related information and acts as the primary entity.

Wallet Class

Attributes:

- walletId: int
- balance: double
- Methods:
 - addMoney()
 - deductMoney()

Each user has one wallet, and this class manages wallet operations.

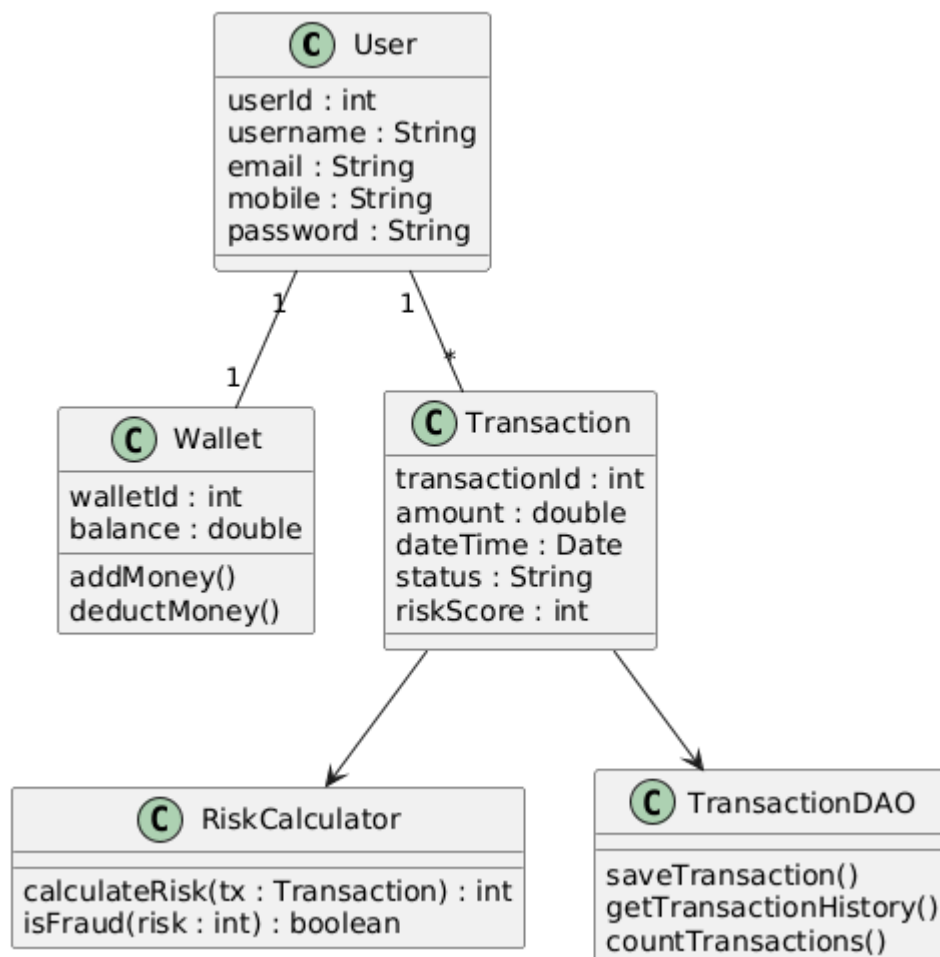


Fig.4.3: Class Diagram

Transaction Class

Attributes:

- transactionId: int
- amount: double
- dateTime: Date
- status: String
- riskScore: int

This class stores transaction details along with fraud-related information.

Risk Calculator Class

Methods:

- calculateRisk (tx: Transaction): int
- isFraud (risk : int) : boolean

This class is responsible for calculating risk scores and identifying fraudulent transactions.

TransactionDAO Class

Methods:

- saveTransaction()
- getTransactionHistory()
- countTransactions()

This class handles database operations related to transactions.

Relationships

- One User → One Wallet
- One User → Multiple Transactions
- Transaction interacts with RiskCalculator
- TransactionDAO manages transaction data

This diagram provides a clear understanding of system structure and object relationships.

4.3.3 Sequence Diagram

The Sequence Diagram shows how different components interact with each other in a time-ordered sequence.

Flow Explanation:

1. The **User** enters username and password on the login page
2. The request is sent to the Auth Servlet for validation
3. The system checks user details in the Database
4. If valid, login is successful
5. The user enters receiver details and amount
6. The request is sent to the Transaction Servlet
7. The system checks wallet balance from the database
8. If balance is sufficient, the system proceeds
9. The transaction is sent to the Risk Calculator
10. The system calculates the Risk Score

Decision Flow:

- If $\text{risk} \geq \text{threshold} \rightarrow$ Transaction is blocked
- If $\text{risk} < \text{threshold} \rightarrow$ Transaction is approved

The result is stored in the Database

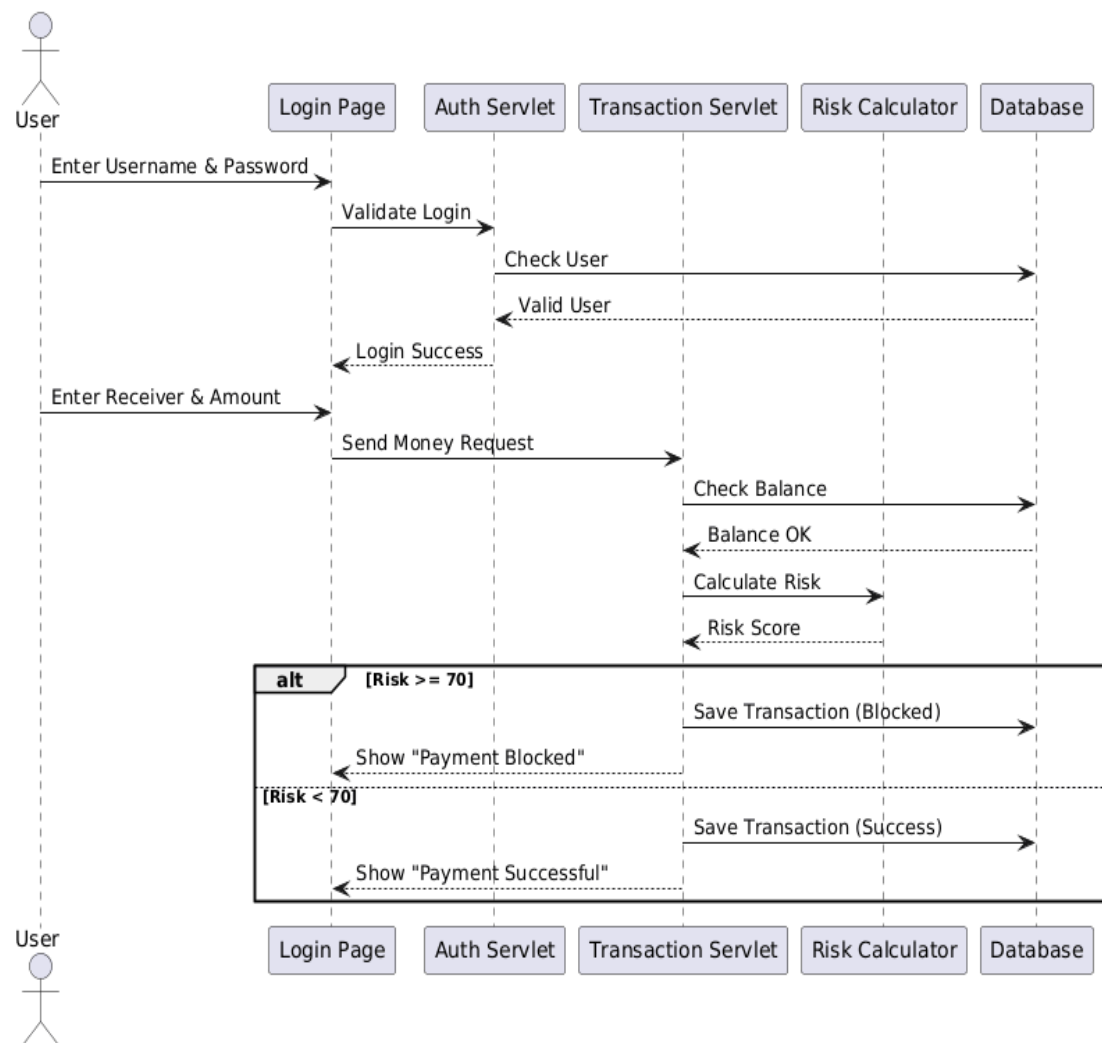


Fig.4.4: Sequence Diagram

The system displays:

- “Payment Successful” OR
- “Payment Blocked”

This diagram clearly shows how real-time fraud detection is integrated into transaction processing.

4.3.4 Activity Diagram

The Activity Diagram represents the workflow of the system and shows step-by-step execution of a transaction process.

Workflow:

1. User logs into the system
2. User selects “Send Money” option
3. User enters receiver details and amount
4. System checks wallet balance
 - If insufficient → Show error message
 - If sufficient → Proceed further

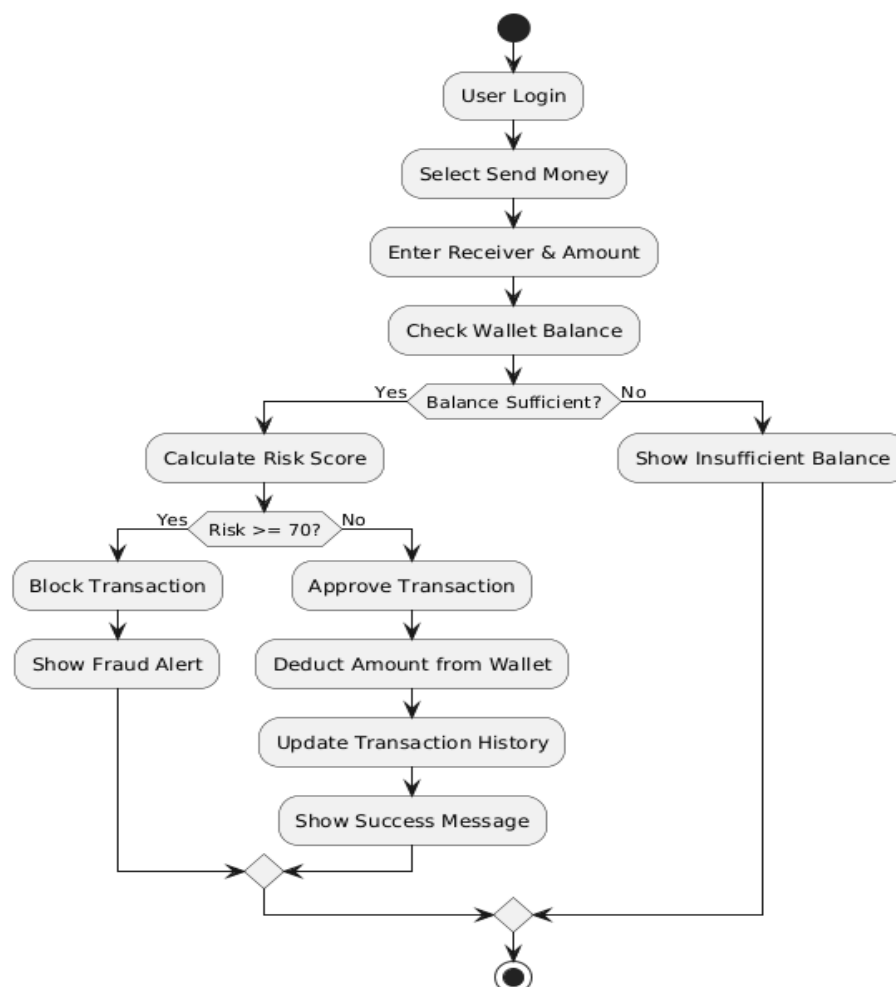


Fig.4.5: Activity Diagram

5. System calculates Risk Score

Decision Making:

- If risk $\geq$ threshold $\rightarrow$ Block Transaction $\rightarrow$ Show Fraud Alert
- If risk $<$ threshold $\rightarrow$ Approve Transaction $\rightarrow$ Deduct amount from wallet
 $\rightarrow$ Update transaction history $\rightarrow$ Show success message

6. Process ends

4.3.5 ER Diagram

The Entity Relationship (ER) Diagram represents the structure of the database and shows how different entities are related to each other in the system. It helps in designing the database schema by defining entities, attributes, and relationships.

In this project, the main entities are User, Wallet, Transaction, and Fraud Check.

Entities and Attributes

1. User Entity

The **User** entity stores all user-related information.

Attributes:

- user_id: int (Primary Key)
- username: string
- email: string
- mobile: string
- password: string

This entity represents registered users who perform transactions in the system.

2. Wallet Entity

The **Wallet** entity manages the financial balance of each user.

Attributes:

- wallet_id: int (Primary Key)
- balance: double

Each user is associated with one wallet, which stores the available balance for transactions.

3. Transaction Entity

The **Transaction** entity stores details of all financial transactions.

Attributes:

- transaction_id: int (Primary Key)
- amount: double
- date_time: datetime
- status: string
- risk_score: int

This entity records every transaction along with its fraud risk evaluation status.

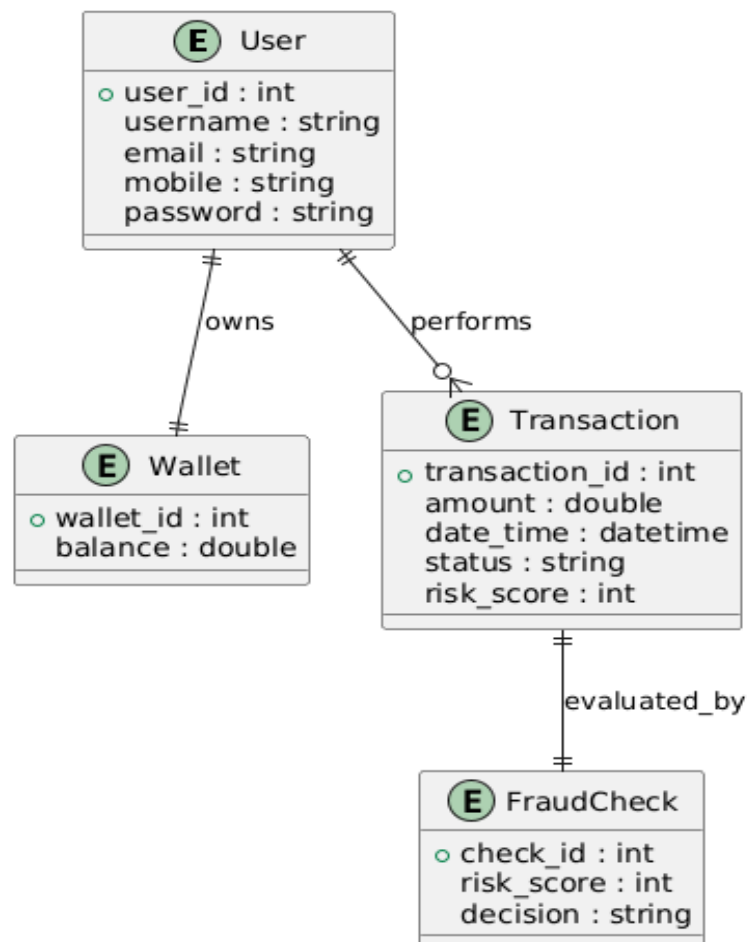


Fig.4.6: ER Diagram

4. Fraud Check Entity

The **Fraud Check** entity is responsible for storing fraud analysis results.

Attributes:

- check_id: int (Primary Key)
- risk_score: int
- decision: string (Approve / Block)

This entity determines whether a transaction is safe or fraudulent.

Relationships

- **User → Wallet (owns)**

Each user owns exactly one wallet (One-to-One relationship).

User → Transaction (performs)

- A user can perform multiple transactions (One-to-Many relationship).

- **Transaction → Fraud Check (evaluated by)**

Each transaction is evaluated by the fraud detection system (One-to-One relationship).

These UML diagrams collectively provide a complete understanding of the system

- Use Case Diagram → User interaction
- Class Diagram → System structure
- Sequence Diagram → Interaction flow
- Activity Diagram → Workflow

They clearly demonstrate how the system integrates real-time fraud detection with secure transaction processing, ensuring reliability and security.

CHAPTER – 5

SYSTEM CODING AND TESTING

5. SYSTEM CODING & TESTING

5.1 Introduction

System Coding and Testing is an essential phase in the Software Development Life Cycle (SDLC), where the designed system is converted into a working application through programming and integration of modules. This phase focuses on implementing system functionalities, ensuring correctness, and validating whether the system meets the specified requirements.

In this project, the Real-Time Fraud Detection System for Online Financial Transactions is developed using Java-based web technologies including JSP, Servlets, and MySQL for developing a complete transaction processing and fraud detection system. The coding process involves implementing key modules such as user authentication, wallet management, transaction processing, real-time fraud detection, and transaction monitoring.

Testing plays a crucial role in ensuring that the system functions correctly, securely, and efficiently. Various testing techniques are applied to verify system performance, detect errors, and ensure that fraudulent transactions are accurately identified and handled. This phase ensures a smooth and reliable user experience.

5.2 Implementation

The implementation phase involves developing the system based on the design specifications and integrating all modules into a fully functional application.

The system is implemented as a dynamic web-based application using a three-tier architecture:

1. Presentation Layer (Front-End)

The front-end is developed using HTML, CSS, and JavaScript, providing a user-friendly interface for users to register, log in, manage their wallet, and perform transactions. The interface is designed to be simple, responsive, and easy to use.

2. Application Layer (Backend)

The backend is implemented using JSP and Servlets, which handle the core business logic of the system. This layer manages functionalities such as:

- User authentication and session management
- Wallet operations (add money, check balance)
- Transaction processing and validation
- Integration of fraud detection logic

The Fraud Detection Module is integrated into this layer, where each transaction is analysed in real time using parameters like amount, frequency, and transaction patterns. A risk score is calculated, and the system decides whether to approve or block the transaction.

3. Data Layer (Database)

The database is implemented using MySQL, where all system data such as user details, wallet information, and transaction history are stored.

JDBC (Java Database Connectivity) is used to connect the application with the database, enabling efficient data storage and retrieval.

System Integration

All modules such as authentication, wallet management, transaction processing, and fraud detection are developed independently and then integrated to ensure smooth communication between components.

When a user initiates a transaction:

1. The system validates user credentials
2. Checks wallet balance
3. Performs fraud detection analysis
4. Approves or blocks the transaction
5. Updates database and displays result

The system is designed to be secure, scalable, and efficient, capable of handling multiple users and real-time transactions simultaneously.

5.3 About Java

Java is a high-level, object-oriented programming language widely used for developing web applications, enterprise systems, and mobile applications. It is

platform-independent, meaning that Java programs can run on any system that supports the Java Virtual Machine (JVM).

Java provides strong support for web development through technologies such as JSP, Servlets, and JDBC, which are used in this project. Its robustness, security features, and extensive libraries make it an ideal choice for building scalable and reliable applications.

5.3.1 History of Java

Java was developed by James Gosling and his team at Sun Microsystems in the early 1990s. It was initially designed for embedded systems but later evolved into a powerful programming language for web and enterprise applications.

Java was officially released in 1995 with the principle of “Write Once, Run Anywhere (WORA)”, which allows Java programs to run on any platform without modification. Over time, Java has undergone several updates, introducing new features, improved performance, and enhanced security.

Today, Java is widely used in various domains such as web development, mobile applications (Android), cloud computing, and enterprise systems.

5.3.2 Advantages of Java

Java offers several advantages that make it suitable for developing this project:

- **Platform Independence:** Java programs can run on any system with JVM support
- **Object-Oriented:** Promotes modular and reusable code
- **Secure:** Provides built-in security features such as bytecode verification and secure APIs
- **Robust:** Strong memory management and exception handling
- **Scalable:** Suitable for large-scale applications
- **Multithreading Support:** Allows concurrent execution of tasks
- **Rich Libraries:** Provides extensive APIs for database, networking, and web development

These advantages make Java an ideal choice for developing secure, scalable, and reliable financial transaction systems.

5.3.3 JDBC (Java Database Connectivity)

JDBC is a Java API used to connect and interact with databases. It provides methods for executing SQL queries and managing database operations.

In this project, JDBC is used to:

- Store user details during registration
- Retrieve data for login authentication
- Insert and update appointment records
- Manage prescription and consultation data

JDBC acts as a bridge between the application and the database, ensuring efficient data communication. It supports operations such as INSERT, UPDATE, DELETE, and SELECT, which are essential for managing system data.

5.3.4 JSP (Java Server Pages)

JSP is a server-side technology used to create dynamic web pages. It allows embedding Java code within HTML to generate dynamic content.

In this project, JSP is used to:

- Display user interfaces such as login, registration, and dashboard pages
- Show dynamic data such as doctor availability and appointment details
- Interact with Servlets for processing requests

JSP simplifies web development by separating presentation logic from business logic, making the application more organized and maintainable.

5.3.5 Servlets

Servlets are Java programs that run on the server and handle client requests and responses. They act as the controller in the application.

In this project, Servlets are used to:

- Process user requests (login, registration, booking)
- Handle form submissions
- Perform validation and business logic
- Communicate with the database using JDBC

Servlets ensure efficient request handling and provide better performance compared to traditional CGI-based applications. They play a crucial role in connecting the front-end (JSP) with the backend (database).

5.4 Sample Code

register.jsp

```
<!DOCTYPE html>
<html>
<head>
  <title> Register</title>
  <link rel="stylesheet" href="style.css">
  <link href="https://fonts.googleapis.com/icon?family=Material+Icons"
rel="stylesheet">
</head>
<body>

<div class="container">
  <h2>
    <span class="material-icons">person_add</span>
    Create Account
  </h2>

  <form action="RegisterServlet" method="post">
    <label>Username</label>
    <input type="text" name="username" required>

    <label>Email</label>
    <input type="email" name="email" required>

    <label>Mobile</label>
    <input type="text" name="mobile" required>

    <label>Password</label>
    <input type="password" name="password" required>

    <button type="submit">
      <span class="material-icons">how_to_reg</span> Register
```

```
        </button>
    </form>

    <p style="text-align:center;">
        Already registered? <a href="/login.jsp">Login</a>
    </p>
</div>
</body>
</html>
```

login.jsp

```
<!DOCTYPE html>
<html>
<head>
    <title> Login</title>
    <link rel="stylesheet" href="style.css">
    <link          href="https://fonts.googleapis.com/icon?family=Material+Icons"
rel="stylesheet">
</head>
<body>
<div class="container">
    <h2>
        <span class="material-icons">account_circle</span><br>
        Login
    </h2>
    <form action="LoginServlet" method="post">
        <label>Username</label>
        <input type="text" name="username" required>

        <label>Password</label>
        <input type="password" name="password" required>
        <button type="submit">
            <span class="material-icons">lock</span> Login
        </button>
    </form>

    <p style="text-align:center;">
```

```
        New user? <a href="register.jsp">Create account</a>
    </p>
</div>
</body>
</html>
```

DBConnect.java

```
package util;

import java.sql.Connection;
import java.sql.DriverManager;

public class DBConnection {
    private static Connection con;
    public static Connection getConnection() {
        try {
            if (con == null || con.isClosed()) {
                // 1 Load MySQL Driver
                Class.forName("com.mysql.cj.jdbc.Driver");
                // 2 Database details (CHANGE PASSWORD ONLY)
                String url = "jdbc:mysql://localhost:3306/fraud_app";
                String username = "root";
                String password = "2004"; // change this
                // 3 Create connection
                con = DriverManager.getConnection(url, username, password);
                System.out.println("Database connected successfully");
            }
        } catch (Exception e) {
            System.out.println("Database connection failed");
            e.printStackTrace();
        }
        return con;
    }
}
```

AddMoneyServlet.java

```
package controller;
```

```
import jakarta.servlet.ServletException;  
import jakarta.servlet.http.HttpServlet;  
import jakarta.servlet.http.HttpServletRequest;  
import jakarta.servlet.http.HttpServletResponse;  
import model.User;  
import service.WalletService;  
import java.io.IOException;
```

```
public class AddMoneyServlet extends HttpServlet {
```

```
    /**
```

```
        *
```

```
    */
```

```
    private static final long serialVersionUID = 1L;
```

```
    @Override
```

```
    protected void doPost(HttpServletRequest req, HttpServletResponse res)  
        throws ServletException, IOException {
```

```
        User user = (User) req.getSession().getAttribute("user");
```

```
        if (user == null) {  
            res.sendRedirect("login.jsp");  
            return;  
        }
```

```
        double amount = Double.parseDouble(req.getParameter("amount"));  
        WalletService walletService = new WalletService();  
        walletService.addBalance(user.getUserId(), amount);  
        res.sendRedirect("DashboardServlet");
```

```
    }  
}
```

FraudAnalysisDAO.java

```
package dao;

import java.sql.*;
import util.DBConnection;

public class FraudAnalysisDAO {

    public void save(int transactionId, int riskScore, String reason) throws Exception
    {
        String sql = ""
            INSERT INTO fraud_analysis(transaction_id,risk_score,fraud_reason)
            VALUES(?,?,?)
            "";
        PreparedStatement ps = DBConnection.getConnection().prepareStatement(sql);
        ps.setInt(1, transactionId);
        ps.setInt(2, riskScore);
        ps.setString(3, reason);
        ps.executeUpdate();
    }
}
```

FraudRiskEngine.java

```
package service;

import model.FraudResult;
import model.Transaction;

public class FraudRiskEngine {

    public FraudResult calculateRisk(
        Transaction tx,
        double userAvgAmount,
        int txCount1Min,
        int txCount5Min,
        boolean newDevice,
        boolean newLocation,
        boolean unusualTime,
```

```
        double mlProbability
    ) {

        int amountRisk = 0;
        int velocityRisk = 0;
        int behaviorRisk = 0;
        int locationRisk = 0;
        int deviceRisk = 0;

        StringBuilder reason = new StringBuilder();

        /* □ Feature 1: Dynamic Amount Anomaly */
        if (tx.getAmount() > 3 * userAvgAmount) {
            amountRisk += 25;
            reason.append("High amount deviation, ");
        }
        /* □ Feature 2: Transaction Velocity */
        if (txCount1Min > 3) {
            velocityRisk += 30;
            reason.append("Multiple rapid transactions, ");
        }
        /* □ Feature 3: User Behaviour Profiling */
        if (unusualTime) {
            behaviorRisk += 15;
            reason.append("Unusual transaction time, ");
        }
        /* □ Feature 4: Receiver Location Awareness */
        if (newLocation && tx.getAmount() > userAvgAmount) {
            locationRisk += 20;
            reason.append("New receiver location, ");
        }
        /* □ Feature 5: Device Change */
        if (newDevice) {
            deviceRisk += 20;
            reason.append("New device detected, ");
        }
        /* □ Final Risk Score */
        int finalRisk =
```

```
        (int) (mlProbability * 50)
        + amountRisk
        + velocityRisk
        + behaviorRisk
        + locationRisk
        + deviceRisk;

    return new FraudResult(finalRisk, reason.toString());
}
}
```

Note: The current system implementation uses the RiskCalculator class based on rule-based logic for fraud detection. Any advanced or machine learning-based components mentioned are considered future enhancements and are not part of the current implementation.

RiskCalculator.java

```
package util;

import dao.TransactionDAO;
import model.Transaction;

public class RiskCalculator {
    private static final TransactionDAO dao = new TransactionDAO();
    public static int calculateRisk(Transaction tx) {

        int risk = 0;
        int userId = tx.getUserId();
        double amount = tx.getAmount();
        try {
            /* □ RULE 1: Amount > 50,000 */
            if (amount > 50000) {
                return 100; // instant block
            }
            /* □ RULE 2: Count 50k transactions today */
            int count50kToday =
                dao.countExactAmountToday(userId, 50000);
            if (amount == 50000 && count50kToday >= 2) {
                return 100;
            }
        } catch (Exception e) {
            // Log the exception
        }
    }
}
```

```
    }
    /* □ RULE 3: Daily total limit */
    double todayTotal =
        dao.getTodayTotalAmount(userId);
    if (todayTotal + amount > 100000) {
        return 100;
    }
    /* □ RULE 4: Velocity (5 seconds) */
    int txLast5Sec =
        dao.countTransactionsLastSeconds(userId, 10);
    if (txLast5Sec >= 2) {
        return 100; // block
    }
    if (txLast5Sec == 1) {
        risk += 40; // suspicious
    }
    /* □ RULE 5: High amount behaviour */
    if (amount >= 30000) {
        risk += 30;
    }
} catch (Exception e) {
    e.printStackTrace();
    return 100; // fail-safe
}
return risk;
}

public static boolean isFraud(int risk) {
    return risk >= 70;
}
}
```

UserService.java

```
package service;
import dao.UserDAO;
import dao.WalletDAO;
import model.User;
public class UserService {

    private final UserDAO userDAO = new UserDAO();
```

```
private final WalletDAO walletDAO = new WalletDAO();

// Register new user
public boolean register(User user) {
    try {
        int userId = userDAO.register(user);

        if (userId > 0) {
            // Create wallet for new user
            walletDAO.createWallet(userId);
            return true;
        }
        return false;
    } catch (Exception e) {
        e.printStackTrace();
        return false;
    }
}
}
```

5.5 Software Testing Strategies

A software testing strategy defines a systematic approach to ensure that the developed system meets the required quality standards, performs correctly, and provides secure and reliable functionality. It involves planning, designing, and executing test cases in a structured manner to identify and eliminate errors at different stages of development.

In this project, the testing strategy follows the principles of Verification and Validation (V&V):

- Verification refers to the process of ensuring that the system is developed correctly according to design specifications. It answers the question: *“Are we building the system right?”*
- Validation ensures that the developed system satisfies user requirements and performs as expected in real-world scenarios. It answers the question: *“Are we building the right system?”*

For the Real-Time Fraud Detection System for Online Financial Transactions, testing is performed at multiple levels to ensure the correctness, security, and reliability of functionalities such as user authentication, wallet management, transaction processing, fraud detection, and transaction history tracking.

The testing strategy includes:

Step-by-step execution of modules:

Each module such as login, wallet operations, transaction processing, and fraud detection is tested individually to ensure proper functionality.

Continuous testing during development:

Testing is performed alongside coding to detect and fix errors early, reducing complexity and improving system quality.

Validation of fraud detection logic:

Transactions are tested under different conditions (normal, high-value, frequent transactions) to ensure accurate risk calculation and correct decision-making (approve/block).

Integration testing between modules:

Ensures smooth interaction between components such as frontend, backend, fraud detection engine, and database.

- **Security testing:**

Verifies that the system protects user data, prevents unauthorized access, and handles sensitive financial information securely.

- **Error detection and handling:**

The system is tested for invalid inputs, insufficient balance, and abnormal transaction patterns to ensure proper error handling.

- **Performance testing:**

Ensures the system can handle multiple users and real-time transactions efficiently without delays.

This structured testing approach ensures that the system is robust, secure, accurate, and ready for real-world deployment, providing reliable fraud detection and safe financial transactions.

5.6 Software Testing Techniques

Software testing techniques are applied to verify that the system functions correctly and meets user requirements. Testing plays a vital role in ensuring system reliability, performance, and security.

In this project, different testing techniques are used to validate individual components as well as the complete system.

5.6.1 Unit Testing

Unit Testing focuses on testing individual components or modules of the system independently to ensure that each module performs its intended function correctly.

In this project, unit testing is performed on the following modules:

- User Registration Module
- Login and Authentication Module

- Wallet Management Module
- Transaction Processing Module
- Fraud Detection Module (Risk Calculation)
- Database Operations (JDBC queries)
- Each module is tested for:
 - Correct input handling (valid and invalid inputs)\
 - Proper validation (e.g., empty fields, incorrect credentials)
 - Accurate processing of logic (e.g., balance deduction, risk calculation)
 - Correct data storage and retrieval from the database

For example:

- In the Login Module, the system verifies that valid credentials allow access, while invalid credentials display appropriate error messages.
- In the Fraud Detection Module, the system checks whether transactions with high-risk scores are correctly identified and blocked.

Unit testing helps in identifying errors at an early stage and ensures that each module functions correctly before integration.

5.6.2 Integration Testing

Integration Testing focuses on verifying the interaction between different modules and ensuring smooth data flow across the system.

In this project, integration testing is performed to verify:

- Interaction between JSP (frontend) and Servlets (backend)
- Data flow between Servlets and Database (JDBC)
- Integration between Wallet Module and Transaction Module
- Integration of Transaction Processing with Fraud Detection Module
- Communication between Fraud Detection Engine and Database

For example:

- When a user sends money, the system checks whether the transaction request is correctly passed from the UI to the backend and processed properly.
- The system verifies whether the fraud detection module receives transaction data, calculates the risk score, and returns the correct decision (approve/block).
- After approval, the system checks whether the wallet balance is updated and the transaction is stored in the database.

Integration testing ensures that all modules work together as a unified system without any errors or data inconsistencies.

5.6.3 Validation Testing

Validation Testing ensures that the complete system meets user requirements and performs all functionalities as expected in real-world scenarios.

In this project, validation testing is performed after integrating all modules into a complete system. It verifies that:

- Users can successfully register and log in
- Users can add money to their wallet and view balance
- Transactions are processed correctly
- Fraudulent transactions are detected and blocked
- Genuine transactions are approved successfully
- Transaction history is stored and displayed accurately

The system is tested with real-world scenarios such as:

- Performing normal transactions (low risk)
- Performing high-value or frequent transactions (high risk)
- Checking system response for insufficient balance
- Verifying fraud detection decisions (approve/block)

Validation testing confirms that the system satisfies user needs, ensures security, and provides a smooth and reliable digital payment experience.

5.6.4 System Testing

System Testing focuses on testing the complete and integrated system to ensure that it works correctly as a whole. It verifies whether all modules function together according to the system requirements and specifications.

In this project, system testing is performed after integrating all modules such as user authentication, wallet management, transaction processing, and fraud detection. The goal is to ensure that the entire system operates smoothly without errors.

Key Areas Tested:

- **End-to-End Transaction Flow:**

The system is tested from user login to transaction completion, including fraud detection and result display.

- **Fraud Detection Accuracy:**

Transactions with different conditions (normal, high amount, repeated transactions) are tested to verify correct approve/block decisions.

- **User Interface Functionality:**

All pages such as registration, login, dashboard, and transaction screens are tested for proper navigation and usability.

- **Database Consistency:**

Ensures that all data (user details, wallet balance, transactions) are stored and retrieved correctly.

TEST CASES

S. No.	TEST CASES	INPUT	EXPECTED RESULT	ACTUAL RESULT	STATUS
1	User Registration	Enter all valid details	User should be registered successfully	Registration successful	Pass
2	User Registration	Missing required fields	Registration should fail	Validation error displayed	Pass
3	User Login	Valid email and password	User dashboard should open	Login successful	Pass
4	User Login	Invalid credentials	Login should fail	Error message displayed	Pass
5	Add Money to Wallet	Enter valid amount	Wallet balance should be updated	Balance updated successfully	Pass
6	Add Money to Wallet	Enter invalid/negative amount	Operation should fail	Error message displayed	Pass
7	Send Money (Valid Transaction)	Valid receiver & sufficient balance	Transaction should be successful	Payment successful	Pass
8	Send Money (Insufficient Balance)	Amount greater than balance	Transaction should fail	Insufficient balance message	Pass
9	Fraud Detection (Low Risk)	Normal transaction (low amount)	Transaction should be approved	Transaction approved	Pass

Table 5.1: Test Case Result

RESULTS

Software results refer to the outputs and outcomes produced by a system after its successful implementation and execution. These results help in evaluating whether the system meets its intended objectives, performs efficiently, and provides reliable functionality in real-world scenarios.

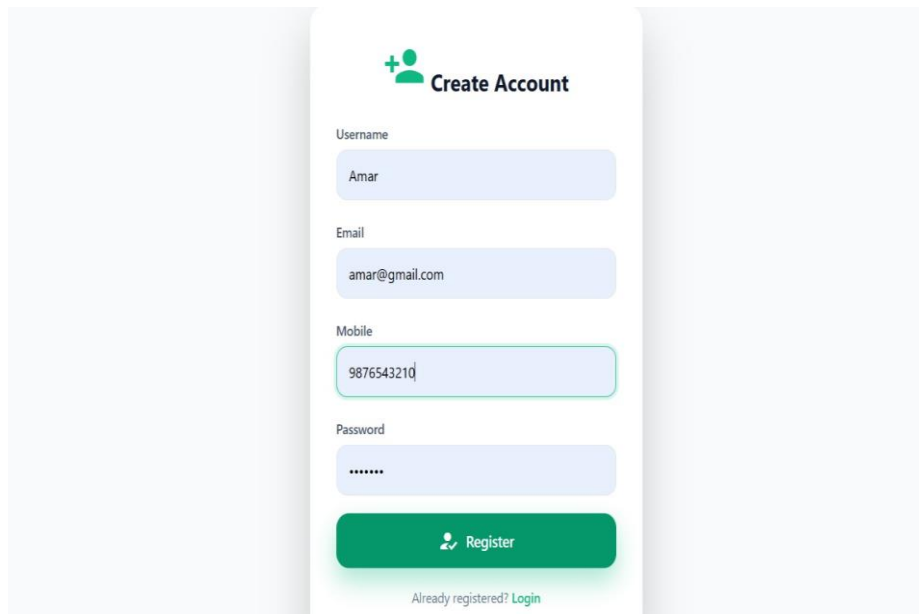
In the case of the Real-Time Fraud Detection System for Online Financial Transactions, the results demonstrate the successful execution of key functionalities such as user registration, secure login, wallet management, transaction processing, and real-time fraud detection. The system generates accurate and immediate outputs, ensuring secure and smooth financial transactions.

The developed application effectively processes user transactions by analyzing various parameters such as transaction amount, frequency, and behaviour patterns. Based on this analysis, the system accurately classifies transactions as safe or suspicious, ensuring that fraudulent transactions are blocked while genuine transactions are processed successfully.

Additionally, the system maintains transaction history and provides real-time feedback to users regarding transaction status (approved or blocked). This improves transparency, enhances security, and builds user trust in the system.

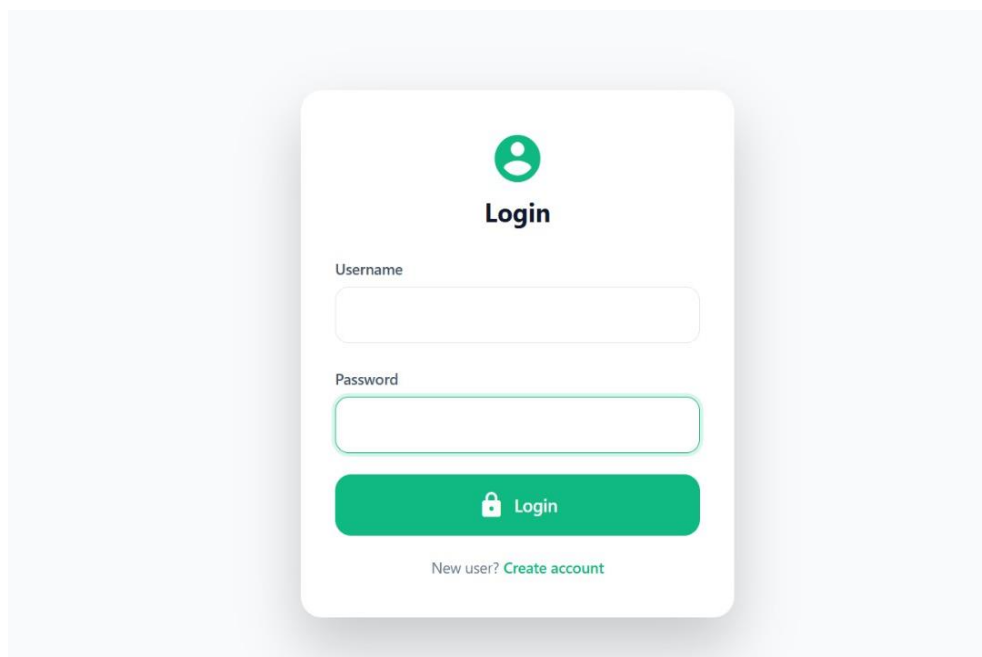
Overall, the results confirm that the system is efficient, reliable, and capable of handling real-time financial transactions while minimizing fraud risks and ensuring a secure digital payment environment.

Output Screens:

A screenshot of a mobile application's registration page. At the top, there is a green plus icon and the text 'Create Account'. Below this, there are four input fields: 'Username' with the value 'Amar', 'Email' with the value 'amar@gmail.com', 'Mobile' with the value '9876543210', and 'Password' with masked characters '*****'. A green button with a white user icon and the text 'Register' is positioned below the password field. At the bottom, there is a link that says 'Already registered? Login'.

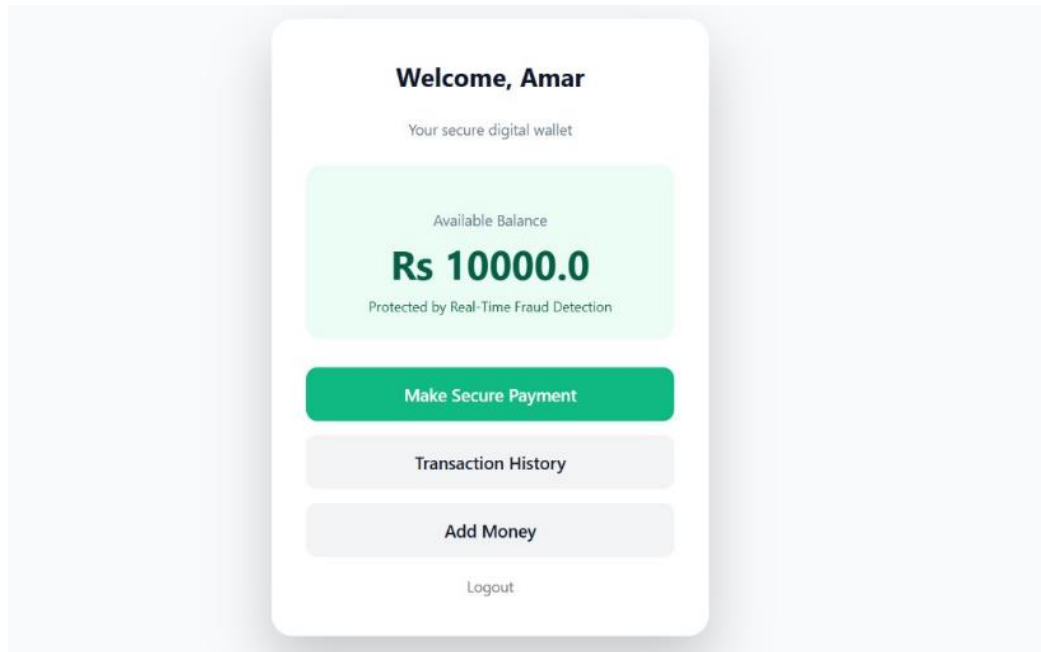
Screen 1: Registration Page

Allows new users to create an account by entering required personal details.

A screenshot of a mobile application's login page. At the top, there is a green user icon and the text 'Login'. Below this, there are two input fields: 'Username' and 'Password'. A green button with a white lock icon and the text 'Login' is positioned below the password field. At the bottom, there is a link that says 'New user? Create account'.

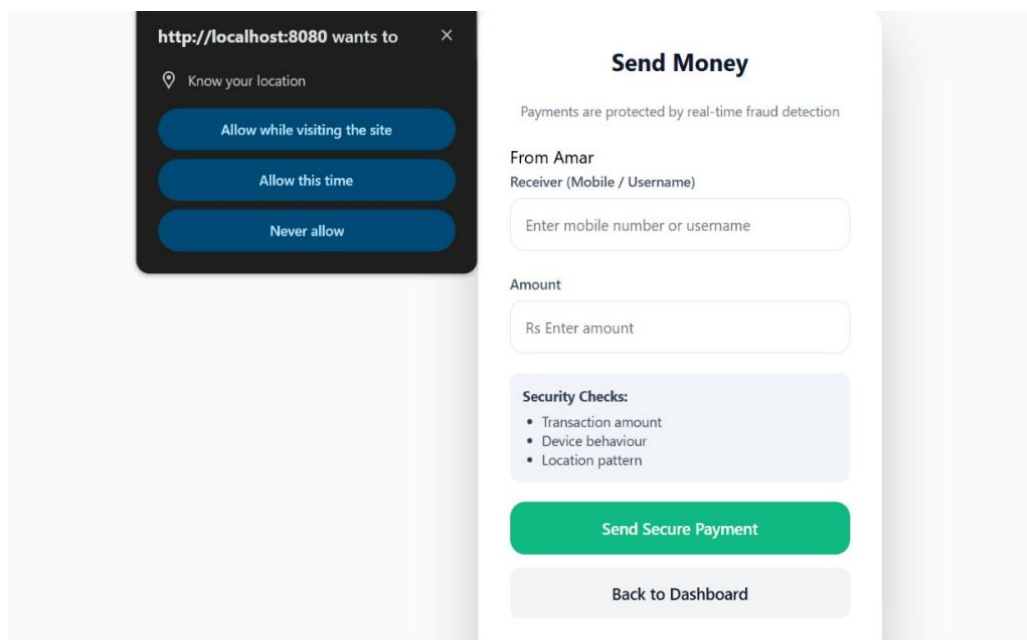
Screen 2 Login Page

Enables registered users to securely log in using their credentials.



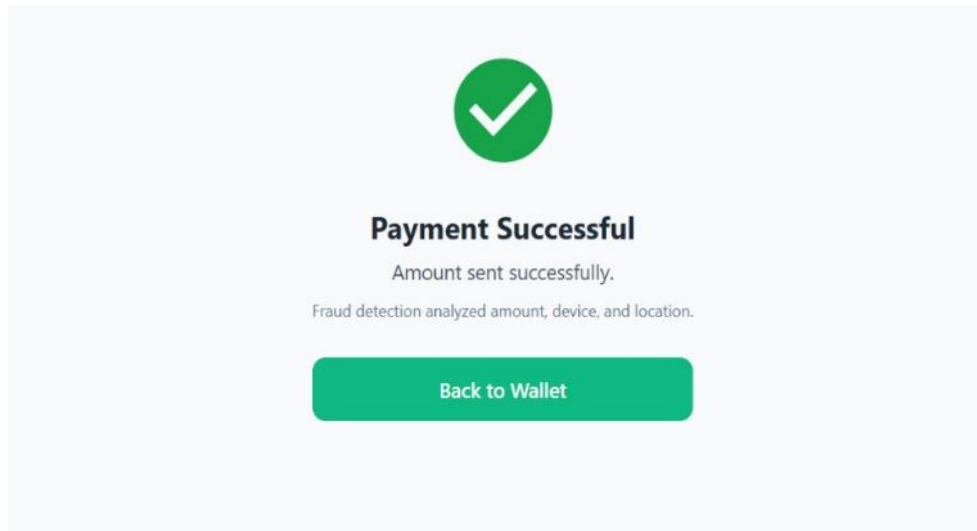
Screen 3 Home page

Displays the user dashboard with options to perform transactions and view account details.



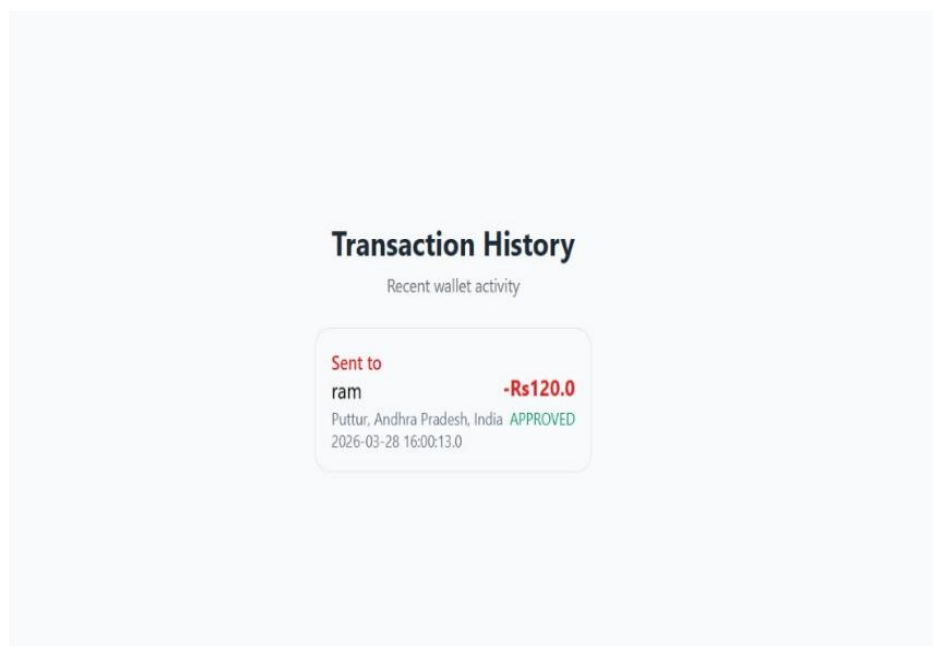
Screen 4 Sending Money

Allows users location to enter receiver details and transfer money from their wallet.



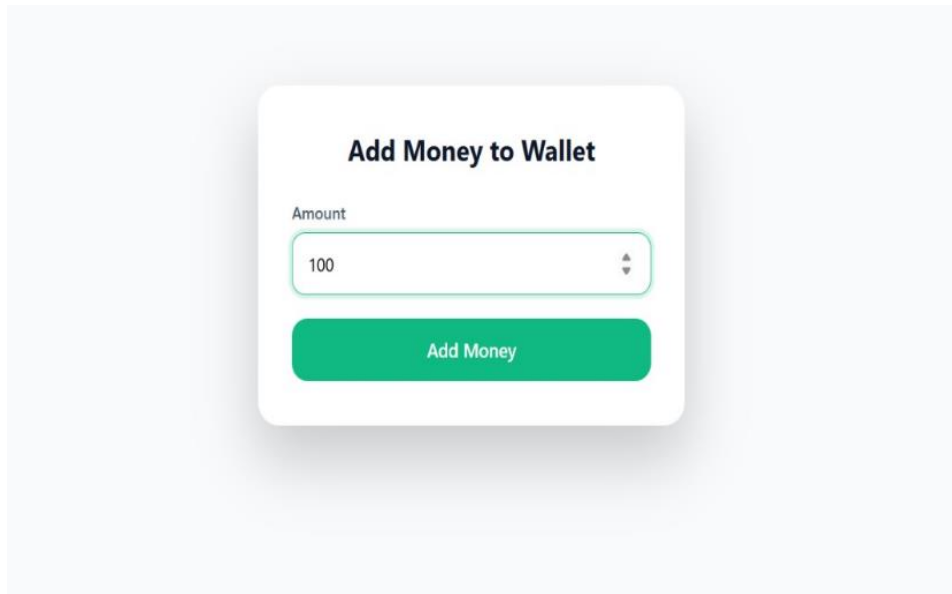
Screen 5: Payment Confirmation

Shows successful transaction details after completing a payment.



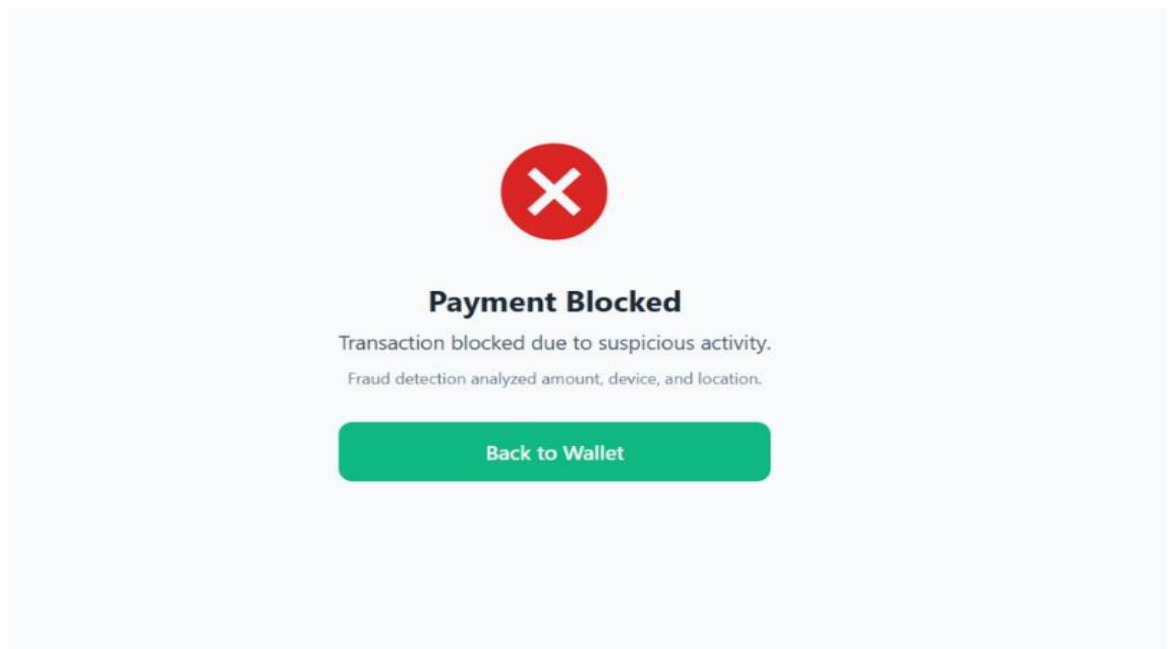
Screen 6: Transaction History

Displays a list of all past transactions with details like amount, date, and status.



Screen 7: Money Wallet

Shows the current wallet balance and allows users to add or manage funds.



Screen 8: Payment Blocked

Displays an alert message when a transaction is blocked due to suspected fraud.

CHAPTER – 6

CONCLUSION AND REFERENCE

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

In conclusion, the Real-Time Fraud Detection System for Online Financial Transactions developed in this project provides a secure and efficient solution to the growing challenges of financial fraud in digital payment systems. The system successfully addresses issues such as unauthorized transactions, delayed fraud detection, and lack of real-time monitoring in traditional systems.

By implementing a dynamic web application using Java-based web technologies including JSP, Servlets, and MySQL, the system enables users to perform financial transactions securely through a digital wallet. The integration of a real-time fraud detection mechanism ensures that every transaction is analyzed based on parameters such as transaction amount, frequency, velocity, and daily limits. Suspicious transactions are identified and blocked instantly, while genuine transactions are processed smoothly.

The project demonstrates that a rule-based real-time fraud detection system can effectively identify and prevent suspicious transactions before financial loss occurs. By analyzing transaction parameters such as amount, frequency, velocity, and daily limits, the system ensures secure and reliable transaction processing. The approach is simple, fast, and suitable for real-time applications without the complexity of advanced algorithms. Overall, the system enhances security, reduces fraud risk, and improves user trust in digital payment platforms.

Overall, the project achieves its objective of delivering a scalable, secure, and intelligent fraud detection system, ensuring safe online financial transactions and contributing to the advancement of secure digital payment technologies.

Limitations of the System –

Although the proposed system provides an effective rule-based approach for detecting fraudulent transactions in real time, it has certain limitations. The system relies on predefined rules and fixed thresholds, which may not adapt to evolving fraud patterns. As a result, it may fail to detect sophisticated or newly emerging fraud techniques.

Additionally, the system may generate false positives in cases where legitimate users perform unusual transactions, such as high-value purchases or sudden changes in transaction behaviour. Since the system does not use machine learning or adaptive algorithms, it lacks the ability to learn from past data and improve detection accuracy automatically.

The system is also limited in handling large-scale real-world datasets, as it has not been tested with high-volume financial transaction environments. Furthermore, integration with external banking systems and APIs is not implemented, which restricts its deployment in real-world financial applications.

The system also demonstrates efficient performance in terms of response time and resource utilization. Since the fraud detection mechanism is based on predefined rules, it requires minimal computational overhead, making it suitable for real-time environments where quick decision-making is critical. This ensures that users experience minimal delay during transaction processing while maintaining system security.

Another significant outcome of this project is the modular architecture adopted during development. Each component, such as user authentication, wallet management, transaction processing, and fraud detection, operates independently yet cohesively. This modular design improves system maintainability, allows easier debugging, and supports future enhancements without affecting the overall system functionality.

From an academic perspective, the project provides a strong foundation in understanding real-time systems, web application development, and secure transaction processing. It also highlights the importance of integrating security mechanisms into application design rather than treating them as an afterthought. This knowledge is essential for developing reliable software systems in real-world scenarios.

6.2 Future Scope

The proposed system can be further enhanced by incorporating advanced technologies and additional features to improve its accuracy, scalability, and overall performance in handling financial transactions securely.

In the future, the system can be extended by:

- **Integrating Machine Learning Algorithms:**

Advanced machine learning models can be used to detect complex fraud patterns and improve prediction accuracy over time.

- **Mobile Application Development:**

Develop Android/iOS applications to enhance accessibility and user convenience.

- **Integration with Banking Systems:**

Enable seamless integration with banking APIs and financial institutions for real-world deployment.

- **Cloud-Based Deployment:**

Deploy the system on cloud platforms to improve scalability, availability, and performance.

- **Big Data Analytics:**

Use large-scale data analysis to identify fraud trends and improve decision-making.

- **Advanced Security Mechanisms:**

Implement encryption techniques, secure APIs, and intrusion detection systems to enhance data protection.

- **AI-Based Behavioral Analysis:**

Analyze user behavior patterns such as location, device usage, and transaction habits for more accurate fraud detection.

- **API integration with banking systems:**

The system can be extended by integrating with real-time banking APIs and payment gateways, enabling direct interaction with financial institutions. This would allow the application to function as a fully operational financial platform rather than a standalone prototype, increasing its practical applicability.

Further research can also explore the use of blockchain technology to ensure secure and transparent transaction records. Additionally, the system can be extended to support various financial services such as credit card transactions, online banking, and e-commerce payments.

Thus, the proposed system has significant potential for future enhancements and can evolve into a comprehensive, intelligent, and highly secure financial transaction system.

REFERENCES

- [1] A.M. Aburbeian, H.I. Ashqar, “Credit Card Fraud Detection Using Enhanced Random Forest Classifier for Imbalanced Data”, in: Daimi, K., Al Sadoon, A. (eds) Proceedings of the 2023 International Conference on Advances in Computing Research (ACR’23), in Lecture Notes in Networks and Systems, Springer, pp. 605-616, 27 May 2023.
- [2] E. Ileberi, Y. Sun, and Z. Wang, “A machine learning based credit card fraud detection using the GA algorithm for feature selection”. in Journal of Big Data 9, 25 February 2022.
- [3] N.S. Alfaiz, S.M. Fati, “Enhanced Credit Card Fraud Detection Model Using Machine Learning”, in Electronics 2022, vol. 11, 21 February 2022. [4] X. Feng, S.-K. Kim, “Novel Machine Learning Based Credit Card Fraud Detection Systems”, in Mathematics 2024, vol. 12, 15 June 2024.
- [4] T. Cayirli and E. Veral, “Outpatient scheduling in health care: A review of literature,” Prod. Operations Manage., vol. 12, no. 4, pp. 519–549, Dec. 2003.
- [5] A. Bansal, H. Garg, “An Efficient Techniques for Fraudulent detection in Credit Card Dataset: A Comprehensive study”, in IOP Conference Series: Materials Science and Engineering, Vol. 1116, April 2021.
- [6] J. K. Afriyie et al., “A supervised machine learning algorithm for detecting and predicting fraud in credit card transactions”, in Decision Analytics Journal, Volume 6, March 2023.
- [7] E. C. Popovici, C. Stalidi, D. A. Teodoras, G. Suciu Jr., I. Cosma, “DeFraudify4ALL: Prototyping and Validation of a System for Fraud Detection with Big Data and Cloud Technology”, to be published in 2024 IEEE 30th International Symposium for Design and Technology in Electronic Packaging (SIITME), Sibiu

Title:

PO MAPPING

S. NO.	PROGRAM OUTCOMES	DESCRIPTION
1	Engineering Knowledge	
2	Problem Analysis	
3	Design/Development of Solutions	
4	Conduct Investigation of Complex Problems	
5	Modern Tool Usage	
6	The Engineer and Society	
7	Environment and Sustainability	
8	Ethics	
9	Individual and Team Work	
10	Project Management and Finance	
11	Life-long Learning	